



M363 Unit 7
UNDERGRADUATE COMPUTING

Software engineering with objects



States and activities

Unit **7**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1613 9

An abstract graphic on the left side of the page, featuring several interlocking, translucent rings or bands that form a spherical, cage-like structure. The rings are arranged in a way that suggests movement and complexity, with some bands appearing to pass through others. The background is a gradient of blue, transitioning from a lighter blue at the top to a darker blue at the bottom, with some subtle ripples or wave patterns visible in the lower half.

CONTENTS

1	Introduction	5
2	Capturing more complex interactions	6
2.1	Conditional message sends in sequence diagrams	6
2.2	Iterative message sends in sequence diagrams	8
2.3	Concurrency	10
2.4	Timing	13
2.5	Summary of section	16
3	Design issues	17
3.1	Preconditions	18
3.2	Strategies for implementing use cases	19
3.3	Summary of section	23
4	An introduction to state machines	24
4.1	Guards	29
4.2	Exceptions	31
4.3	Entry and exit events and actions	32
4.4	The final state	34
4.5	When to use state machines	35
4.6	Summary of section	37
5	Variation of events and actions	39
5.1	Other kinds of event	39
5.2	Looking inside states	42
5.3	A final note on state diagrams	45
5.4	Summary of section	48
6	Packages	49
6.1	Subsystems	52
6.2	Summary of section	55
7	Summary	56
	Index	58

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

The aims of this unit are to:

- ▶ extend your ability to model the dynamic behaviour of a software system;
- ▶ introduce a technique for modelling the life histories of objects;
- ▶ show how model elements can be grouped into packages and subsystems.

To best understand this unit, you should know the important concepts associated with object-oriented software development. You should be familiar with the static models within Unified Modeling Language (UML) that were introduced earlier in this course. Finally, it is assumed that you have already encountered some forms of dynamic modelling, namely the interaction view found within sequence and communication diagrams.

2

Capturing more complex interactions

In Section 3 of *Unit 6*, you saw that it is possible to build sequence diagrams at different degrees of generality. Some designers prefer to have a number of diagrams, each denoting a concrete scenario. Examples might be *jill* checking into the *theRitz* when it has two free rooms, and *jill* checking into the *theRitz* when it has no free rooms. Each diagram is totally unconditional because, starting from a concrete state, the software system can behave in only one way when it receives a message. In effect, a diagram for a concrete scenario specifies a test case for that scenario.

Other designers prefer to put all these cases onto a single diagram that captures a range of possible behaviours. Such a diagram will have to include conditional behaviour, because the message flow will depend on the initial state in some way. The diagrams are more complex, but there are fewer of them. This is an example of moving from specific to more generic diagrams.

In practice, you are likely to use both views, depending on the complexity of each use case and its accompanying scenarios. But you should try to keep your diagrams easy to read and understand. If a particular interaction requires a diagram that is hard to read, it may be a sign that the use case you are modelling should be split into two or more use cases. We recommend that you use generic interactions sparingly, since you can represent branching and joining much better within an activity diagram.

You saw in *Unit 3* how activity diagrams are able to represent a user's routine. They also help to confirm and/or identify use cases. In the next section, we will discuss the benefits of turning your use cases into classes.

2.1

Conditional message sends in sequence diagrams

If you choose to include conditional behaviour on a sequence diagram, you will need some extra notation. You need to show the condition that must be satisfied before a particular message can be sent. In *Unit 3*, you saw how transitions between activities were guarded to constrain a workflow. The same notation can be applied to the messages in an interaction diagram as we will now illustrate. In the hotel check in example (see Figure 6 of *Unit 6*), we made a *Hotel* object responsible for finding a free room for the guest to occupy. Figure 1 shows a more generic sequence diagram that will put a guest into a room only if a free room has been found. The message *accept (jill)* has been prefixed with the guard *[room available]*. You can use the same notation to attach a guard to the messages in a communication diagram.

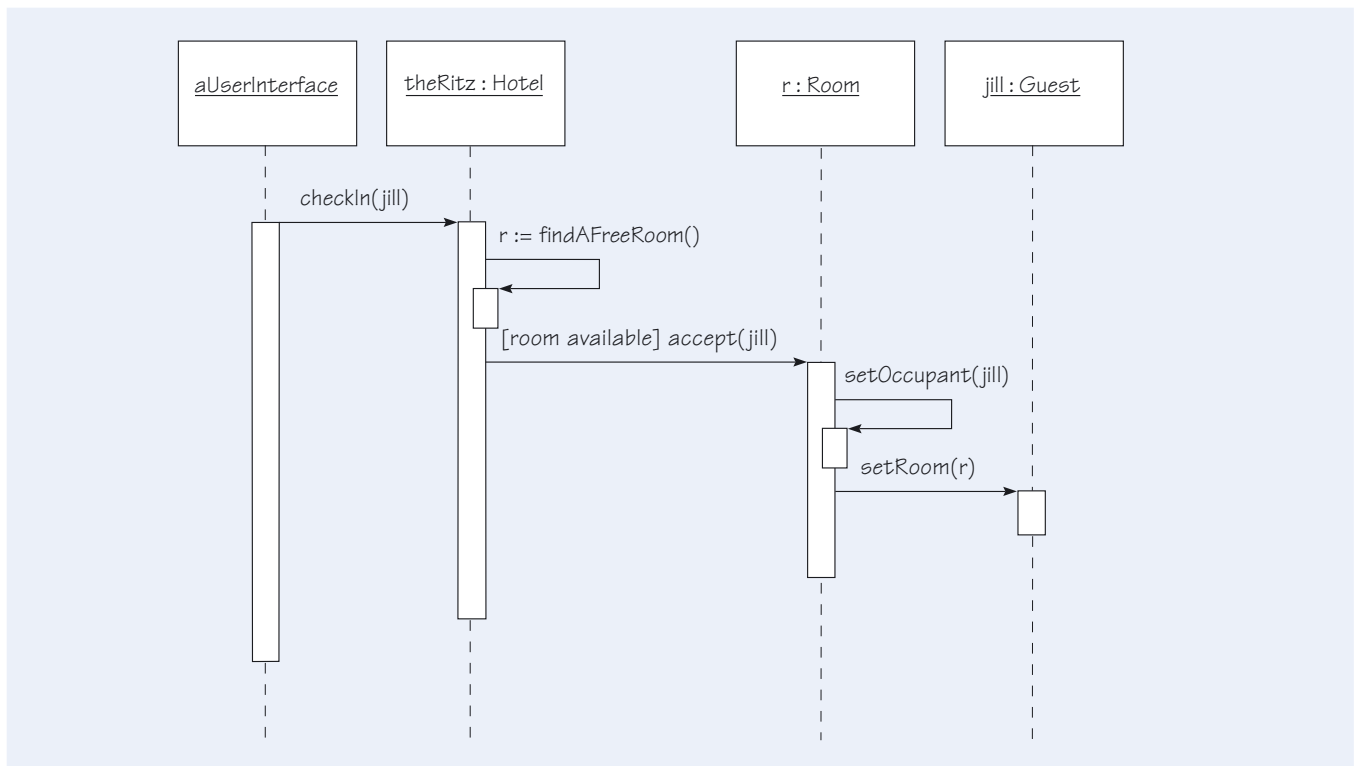


Figure 1 A sequence diagram showing a guard

Even more generality than that shown in Figure 1 is needed to enable you to represent the alternative courses of action required for different scenarios. For example, suppose that a particular room is unoccupied but still needs to be cleaned. Different messages would be sent depending on the state of the *Room* object (as discovered by the operation *findAFreeRoom()*). This is illustrated in Figure 2, overleaf, where the *Room* object *r* performs a different operation depending on whether or not it is ready for occupation. Notice how UML allows you to add a branch to the lifeline of an object in a sequence diagram. Such an interaction cannot be shown on a communication diagram.

If more than one conditional message originates from the same point on the lifeline of an object in a sequential system (a non-concurrent system), the conditions must be mutually exclusive.

In order to avoid complexity, we recommend that you place each scenario into a single sequence diagram. If you attempt to show all the things that could happen in each of the scenarios in a single use case, the resultant sequence diagram can become very crowded – for example, trying to show the maintenance of a room as well as its cleaning on the same generic sequence diagram. Partitioning the scenarios in such a manner also allows you to explore different designs. You may, for example, find a more suitable solution by dealing with maintenance and cleaning checks within the operation *findAFreeRoom*. A room is available for occupation by a guest or it is not – a simplification that meets the precondition that was introduced in *Unit 1*.

UML does not specify the syntax of the conditional expression in a guard. However, whatever syntax you use, it must represent a Boolean expression that results in either true or false, and which involves only attributes available system-wide (for example, time and date), or from the current operation – the attributes of the sender or attributes of any parameters of the invoked operation. You could use English, your chosen programming language or some other notation, such as OCL, as long as some consistency is maintained during the project (at least).

Recall that *r* is simply a placeholder for the reference to the room object found by the *findAFreeRoom()* message. It is not a programming variable. See *Unit 6*.

The return messages have been shown to emphasise the flow of control coming back to the same point in the activation of the sender.

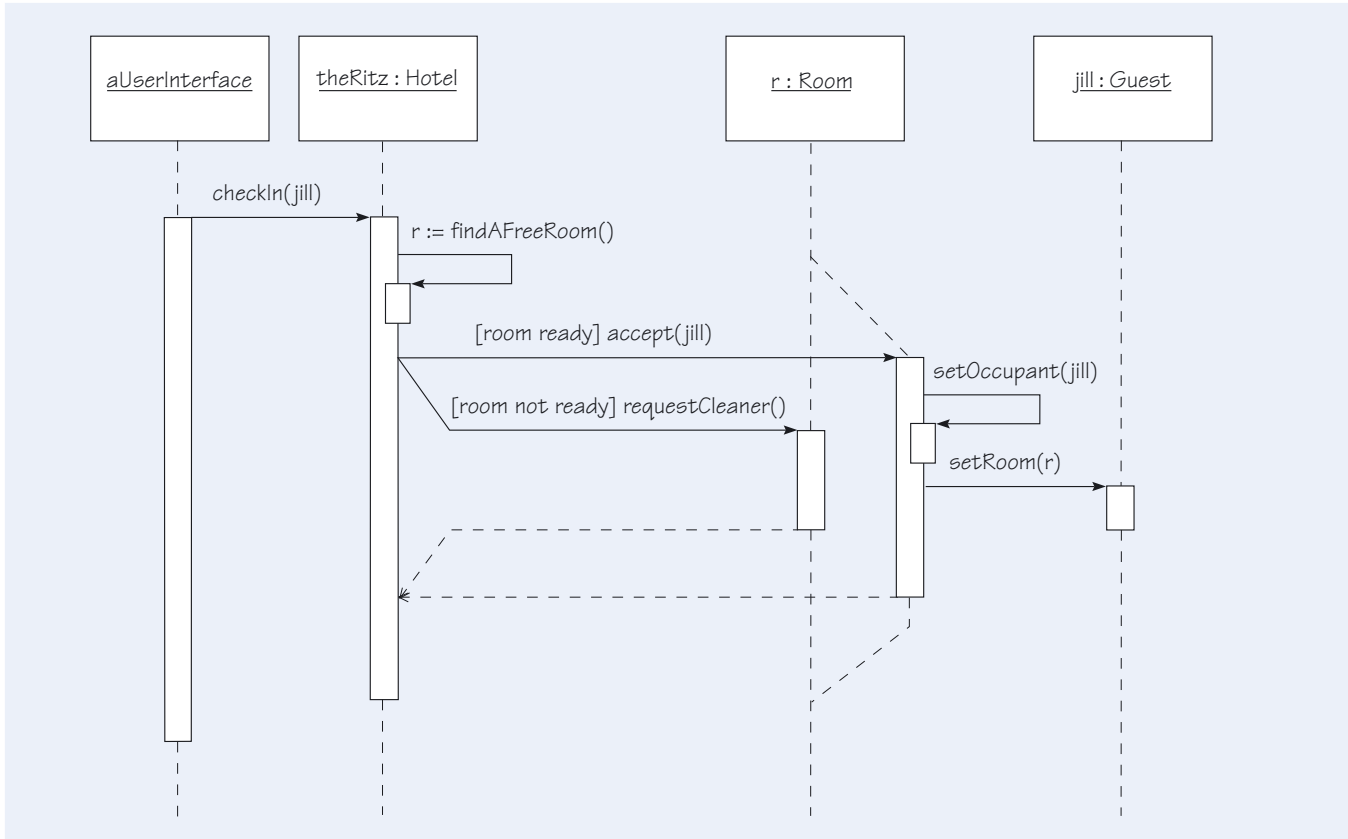


Figure 2 Alternative course of action in a sequence diagram

2.2 Iterative message sends in sequence diagrams

Occasionally you may want to send a sequence of messages to the same receiver. If the messages only differ by some simple parameter, such as a counter, your diagram could become cluttered with repeated messages. UML uses the *** notation for a repeated message send, which may be attached to a guard, generating an iteration clause such as **[i := 1..10]* to send one message ten times. It is assumed that the messages are executed sequentially. Any nested messages in a sequence will be repeated in accordance with the iteration clause that you define.

Figure 3 shows the identical message *moveForward(10)* being sent twice. Each time an instance of the *Car* class receives that message, it sends the message *sound()* to an instance of the *Horn* class.

A more common use of iteration is to make at least one parameter of the message vary between sends. A company might print a year's reports by sending to an instance of a *Department* class the message *printReportForMonth(i)*, using an iteration clause **[i := 1..12]*.

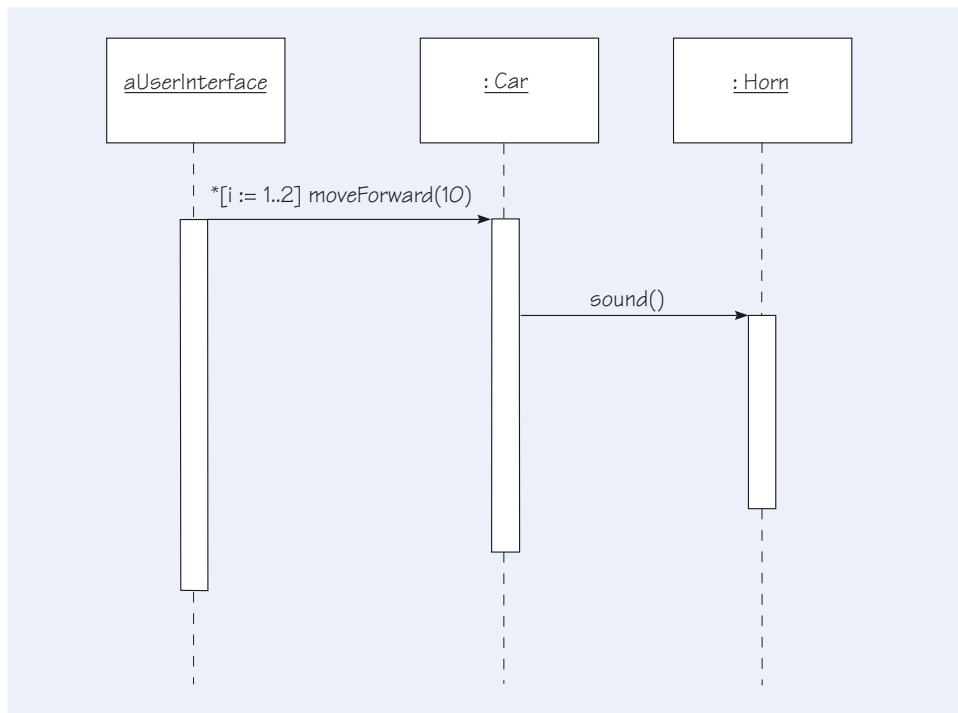


Figure 3 A sequence diagram showing iteration

SAQ 1

- Why must the conditions on a message send be mutually exclusive in a sequential system?
- Figure 2 shows two conditional message sends originating at the same moment on the timeline, meaning that all the guards are evaluated before any message is sent. Could the same behaviour be implemented if the messages (and their conditions) were separated on the time line?
- Suggest a reason why an object's lifeline must branch in cases such as Figure 2.
- Are the examples of sequence diagrams in *Unit 6* generic or instance interaction diagrams?
- How would you change the sequence diagram in Figure 3 to sound the car's horn five times each time it moved forward?

ANSWER.....

- If conditions were not mutually exclusive, more than one condition might be true. This would mean that multiple messages would be sent at once, leading to multiple receiving objects being active at the same time – violating the assumption that the system is sequential and not concurrent.
- Yes, provided that the conditions were mutually exclusive, so that the first message could not affect the state of the receiver before the second message arrived. We might use code such as the following.

```

if (room.ready) room.accept(jill);
if (!room.ready) room.requestCleaner();

```
- The *Room* object will receive different messages depending on which branch is executed in the *Hotel* object. It makes no sense for the messages to arrive at just one lifeline, as that would imply that the *Room* object received all of the messages in a single interaction. Instead, we show a fork in the lifeline of the receiving object to reflect the fork of control in the sending object.

- (d) All the examples in the course text in *Unit 6*, except Figures 12, 17 and 18, are instance interaction diagrams, as they show a specific (concrete) sequence of messages. (Figures 1 and 2 of this unit are not pure generic interaction diagrams either, as they still contain concrete instances of certain classes.)
- (e) To get the horn to sound five times for every forward movement in the sequence diagram, you would add an iteration clause to the second, nested message:
`*[j := 1..5] sound()`.

2.3 Concurrency

Apart from a brief example in an activity diagram in *Unit 3*, concurrency has not arisen so far in this course. We have implicitly made two assumptions. The first assumption is that all objects are **passive**. That is, objects remain inactive until a message arrives. Hence, between processing messages, objects do nothing (they are not active). For example, the notion of an alarm-clock object that does nothing for a long time, and then suddenly sends a message to other objects, cannot be modelled unless the clock is given an external stimulus. The second assumption is that there is a single thread of control. Only one object does any processing at any one time, and an object can never be asked to do two things at once.

Neither of the above assumptions is typically written into a requirements document, so it is interesting to note how they have been introduced into our discussion of the hotel. We collected a set of use cases, and built a class model, but most of the real design took place during the building of sequence diagrams. Since we considered one scenario and built one sequence diagram at a time, it was easy to forget that unless the requirements documents indicated anything to the contrary, any number of use cases might be active at any one time.

During design, you need to bear in mind that when a message is sent to an object, it may already be executing a method for some other client. Therefore, concurrency may need to be included in the design.

UML provides many notations for handling concurrency. The primary one is the ability to designate an object as **active**. An active object is one that owns a thread of control, and can initiate control activity. In Java terms, it is a thread. An **active object** is an instance of an **active class**. Active objects and active classes are shown in UML by a thick black border, as shown in Figure 4. Note that the same notation is used in sequence diagrams: active objects have thick black borders.

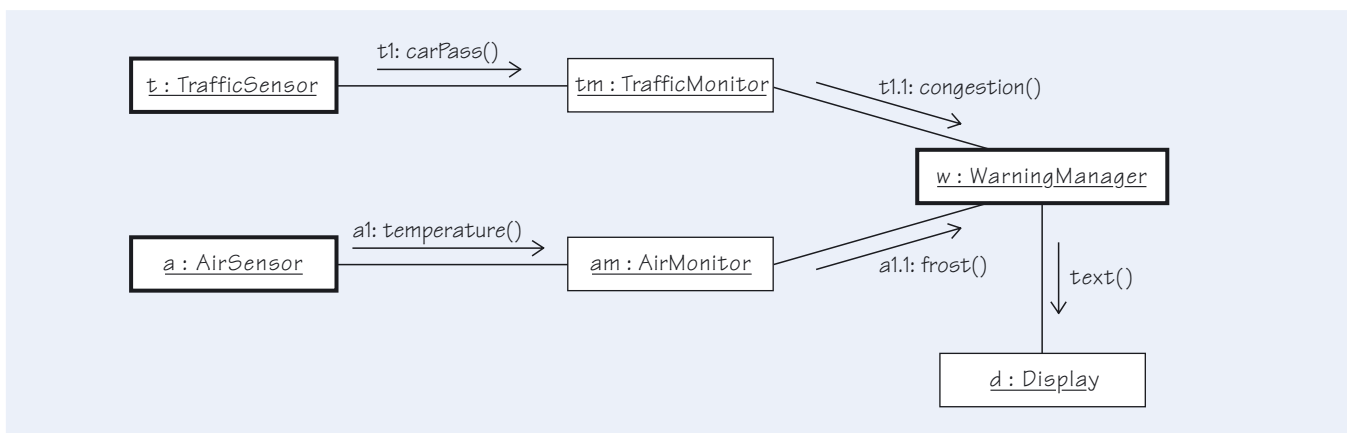


Figure 4 Active objects

The communication diagram in Figure 4 describes a possible warning system, which displays information to motorists about driving conditions. An active *TrafficSensor*, which continually interrogates radar devices and pads in the road, sends messages to a passive *TrafficMonitor*. Similarly, there is an active *AirSensor* that runs continuously and, every now and again, sends messages to a passive *AirMonitor*. Both the *TrafficMonitor* and the *AirMonitor* send messages to an active *WarningManager*. The *WarningManager* filters the warnings it receives; then it updates and reorders them. It also communicates with a passive *Display* to show messages to drivers.

The sequences of messages are numbered using the multi-stage numbering system introduced in Section 4 of *Unit 6*. In addition, each message has a label that identifies the thread that sends it. The *TrafficSensor* *t* is active; therefore it has its own thread of control. All messages sent by *t* have *t* prefixed to the message number. The *TrafficMonitor* is passive, and so will only send messages as a result of receiving them. Consequently, the *congestion()* message is numbered *t1.1*. It is running in the same thread as message *t1*. Similarly, the *frost()* message is numbered *a1.1* because it is in the same thread as message *a1*.

As there is more than one active thread, messages may be sent concurrently. This raises several questions. For example, what should happen if several messages arrive simultaneously at an object? Suppose you have two traffic sensors that simultaneously want to send *carPass()* messages to the *TrafficMonitor*. Will the *TrafficMonitor* be able to cope?

To enable you to answer these questions, UML distinguishes three different situations, as follows.

- 1 The *TrafficMonitor* might not be able to cope with simultaneous activations. It might take the *sequential* approach to synchronisation, insisting that callers somehow synchronise among themselves, and agree to send only one message at any one time. It does not guarantee that simultaneous messages will be handled successfully.
- 2 *TrafficMonitor* guarantees its integrity by making various operations serialised, labelling them *guarded*. Only one such operation will be allowed to be active at any one time for a given instance of *TrafficMonitor*. The other operations are blocked until the first one has completed.
- 3 *TrafficMonitor* guarantees its integrity by making various operations atomic, labelling them *concurrent*. Multiple calls from concurrent threads can happen simultaneously to an instance of *TrafficMonitor*. All of them are able to proceed and follow the correct semantics for any concurrent operation in *TrafficMonitor*.

The unit of blocking is at the object level in the UML.

This is equivalent to the *synchronized* concept in Java.

Passive instances of classes such as *TrafficMonitor* and *WarningManager* are likely to have query operations that do not change the state of an instance; these operations can also be labelled as concurrent, to allow multiple calls to take place. Figure 5 shows a class with synchronisation annotations on the methods. It has marked *frost* and *congestion* with the constraint {*concurrent*} to signify that they are atomic. If they both manipulate some common data structures, the two operations cannot interfere with each other. The *getTotalWarnings* operation has also been marked, presumably because it needs to operate in parallel with all the other operations.

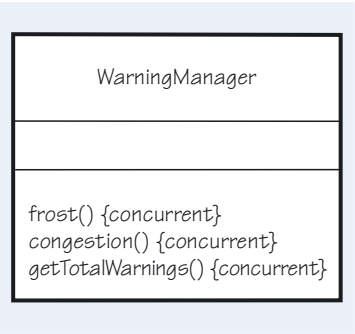


Figure 5 Annotating methods with synchronisation

So far, we have always treated a message send as having the same blocking semantics as a synchronous method call: the sender of a message blocks until the receiver has finished processing the message; when the receiver completes its processing of the message, the sender resumes. This notion can survive in the world of active objects, and is often useful. When an active object sends a message to a passive object, this choice of semantics may be sensible. The active *TrafficSensor* sends the *carPass()* message to the passive *TrafficMonitor*, and it may be perfectly satisfactory that the *TrafficSensor* does nothing until the *TrafficMonitor* has finished processing. If, however, the *TrafficMonitor* took a very long time to process a message, information would be lost if the *TrafficSensor* detected more vehicles than the *TrafficMonitor* could deal with. (This is an example of a problem called fast-sender/ slow-receiver.) Thus it might make sense to turn the *TrafficMonitor* into an active object, so that both sensor and monitor can work in parallel, as shown in Figure 6.

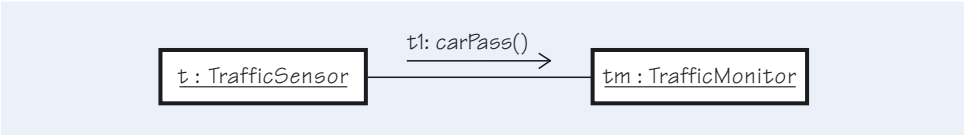


Figure 6 Making an object active

There are several ways of synchronising two active objects that are communicating with each other. One way is to ensure that both threads are at some time simultaneously ready to communicate. This is **rendezvous semantics**. The sender indicates a desire to talk, and the receiver indicates a desire to listen. Whichever of these happens first causes that thread to block until the other has happened, when communication takes place. A consequence is that the receiver can never receive an unexpected message, as it must have indicated a desire to receive something. Rendezvous semantics help solve the fast-sender/slow-receiver problem by permitting parallel processing of a single rendezvous, but the sender (*TrafficSensor* in our case) can still block if the receiver (*TrafficMonitor*) has not finished.

Another way to make two active objects communicate is to use **mailbox semantics**. The sender sends a message, and then carries on processing, regardless of what the receiver is doing. The receiver, operating at its own speed, can then read waiting messages and process them. Information has flowed, but there is no explicit synchronisation. Of course, something must provide the queue of messages (the mailbox), and its queue must be of sufficient size to accommodate the maximum number of messages waiting to be processed by the receiver. This form of synchronisation solves the fast-sender/slow-receiver problem, under the condition that the receiver will empty the queue of messages at a rate sufficient to prevent the queue from overflowing.

Figure 7 shows the variety of arrows that UML uses for different forms of message sending. So far we have always used *synchronous* message sending, in which the

Earlier versions of UML used a half-stick arrowhead to indicate asynchrony. In UML 1.4, the half-stick arrowhead was replaced by the stick arrowhead.

sender blocks until the receiver returns control to it. We have sometimes used the dashed, stick arrow to mark the return from a synchronous message send, though usually we have omitted it in order to save cluttering the diagrams. The stick arrow of an asynchronous send is related to mailbox semantics – the sender carries on processing immediately after sending a message. For this reason, there cannot be a value returned as a result of sending the message. (The recipient could reply later in a separate message.)

Symbol	Message
	procedural or synchronous
	return from a procedural call
	asynchronous

Figure 7 Message styles for interaction diagrams

SAQ 2

How does UML distinguish a synchronous message send from an asynchronous message send? How is a return from a synchronous message send indicated?

ANSWER.....

A synchronous message send has a solid arrowhead and a continuous shaft, whereas an asynchronous message send has a stick arrowhead and a continuous shaft. The return from a synchronous message send is marked by a stick arrowhead with a dashed shaft.

Even though the return from a synchronous message send is itself synchronous, UML uses the stick arrowhead.

2.4 Timing

In our discussion about the behaviour of a set of objects, we have only considered the relative ordering of messages in interactions between them. Sometimes, the time taken to send, process or return messages is important. We can identify timing constraints as Boolean expressions on a sequence or communication diagram using two distinct properties defined in UML:

- ▶ the time at which a message or signal is sent by an object – the *sendTime*;
- ▶ the time at which that message or signal is received by an object – the *receiveTime*.

In concurrent systems, message labels can be used to identify the sequences of messages. Such labels can also be used to identify the sending and receiving of messages. Figure 8 shows two examples of recording timing constraints in a sequence diagram, which is based on the warning system given in Figure 4. The first example (seen on the left of the diagram) records the requirement that the processing time required by *tm* : *TrafficMonitor* plus any message-queuing and transmission times should be less than 5 s (seconds). Figure 8 shows how the message labels, *a* and *b*, are used to identify the start and end points of the message sequence, and the constraint uses the *sendTime* and *receiveTime* properties to give the required times. The timing constraint is from the point that *t* : *TrafficSensor* sends to the point that *w* : *WarningManager* receives, and can therefore be expressed as {*b.receiveTime* – *a.sendTime* < 5 s}. Note that the labels *a* and *b* provide sufficient identification of the

messages. The constraint text need not be placed near the label. That being said, placing the constraint text far away from the label will make the diagram hard to read.

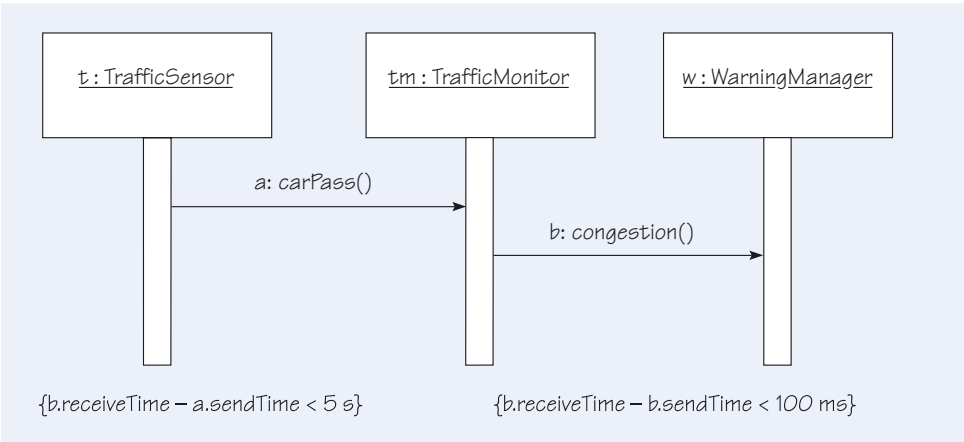


Figure 8 A sequence diagram for a warning system

In the second example, the requirement is a limit on the time taken to send and receive a message. Figure 8 illustrates a timing constraint that limits the time to pass a message between *tm : TrafficMonitor* and *w : WarningManager* to less than 100 ms (milliseconds), namely $\{b.receiveTime - b.sendTime < 100\text{ ms}\}$.

Timing constraints can be found in both procedural and asynchronous interactions. In simple cases, it is acceptable to represent timing constraints on a sequence diagram within braces. In more complex cases, we recommend that you use a UML note. As the number of timing constraints increases, the need for a suitable CASE tool to record them also increases.

The lifeline of an active object is hidden behind its activation box.

SAQ 3

What properties of a message or a signal are used to express a timing constraint?

ANSWER.....

The *sendTime* and *receiveTime* properties of a message or a signal are used to express a timing constraint. In addition, a message label is required to express the time difference between the start and end points, as shown in the examples in Figure 8.



Exercise 1

Figure 9 shows part of a lift controlled through a graphical user interface (GUI). Add timing constraints to show that:

- ▶ a *mouseUp* must occur within 1 ms of a *mouseDown*;
- ▶ two successive *mouseDown* events must be separated by at least 1 s;
- ▶ the transmission time of the message from the button to the lift must be less than 500 ms.

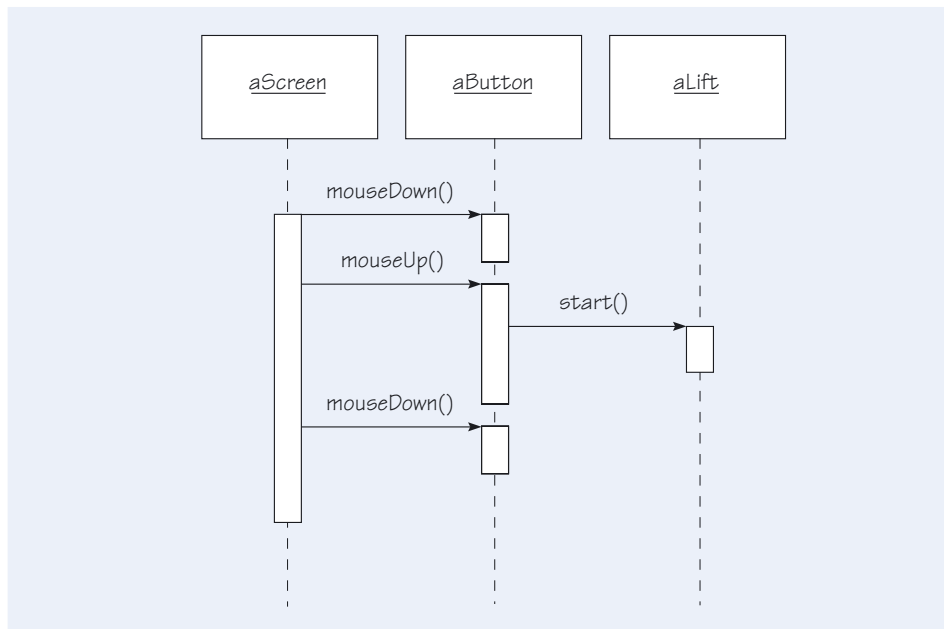


Figure 9 A lift controller

Solution

The first step is to label the messages *mouseDown()* (first occurrence), *mouseUp()*, *start()* and *mouseDown()* (second occurrence) as *a*, *b*, *c* and *d* respectively. Then, we can use the *sendTime* and *receiveTime* for each message to express the timing constraints as follows.

- ▶ A *mouseUp* must occur within 1 ms of a *mouseDown*:
 $\{b.sendTime - a.sendTime < 1\ ms\}$
- ▶ Two successive *mouseDown* events must be separated by at least 1 s:
 $\{d.sendTime - a.sendTime \geq 1\ s\}$
- ▶ The transmission time of the message from the button to the lift must be less than 500 ms:
 $\{c.receiveTime - c.sendTime < 500\ ms\}$

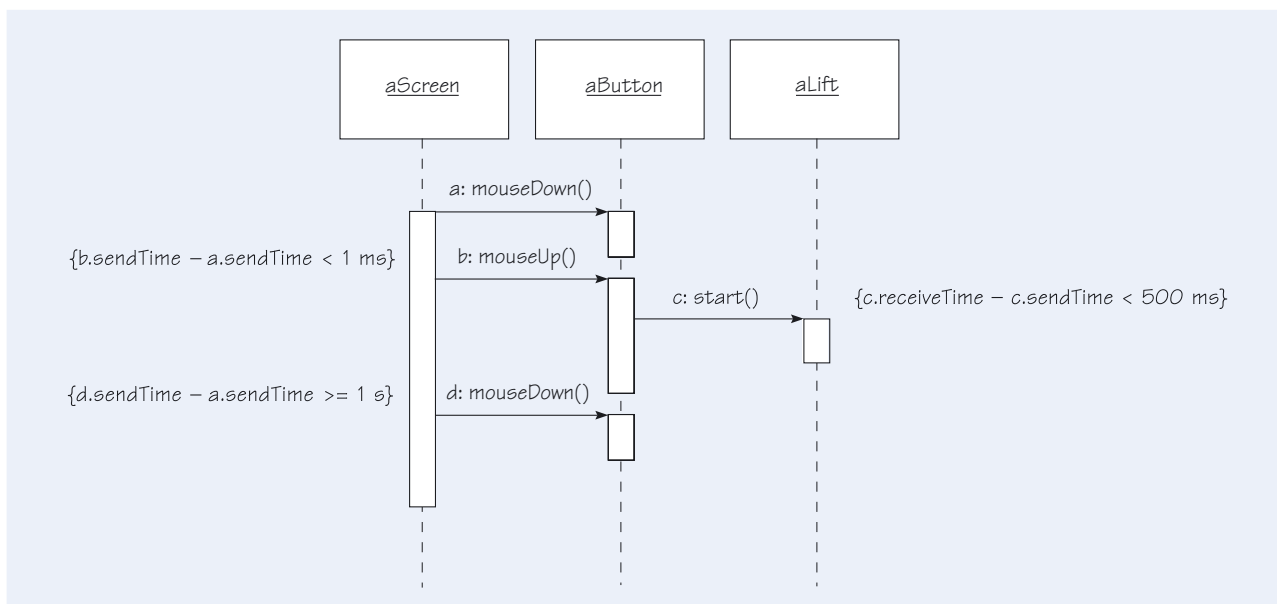


Figure 10 A lift controller with timing constraints

Figure 10 shows the addition of timing constraints to the original sequence diagram.

2.5 Summary of section

This section has discussed how messages should be used to initiate the tasks involved in a use case.

- ▶ UML provides a suitable notation to enable conditional messages to be expressed in sequence diagrams.
- ▶ UML provides a suitable notation for iterative message sending, and for constraining the number of iterations.
- ▶ The use of active objects is a way of expressing concurrency in a design. UML provides a notation to enable synchronous and asynchronous messages to be identified.
- ▶ Timing constraints can be added to sequence diagrams to specify limits that must be satisfied by any implementation of the diagram.

3

Design issues

The interaction diagrams that you encountered in *Unit 6* were used to explore interactions within a business model. Each of them was initiated by a single message coming from a user interface. For the checking in message in the hotel system, one might be tempted to make the interface send a message such as:

`checkIn("Jill Smith")`, passing a name as a string,

or

`checkIn("res23")`, passing some sort of identifier.

The disadvantage of both these messages is that the peculiarities of the interface affect the design of the business model. They are both attempts at identifying objects by their value (using what is called an *external identifier*), but in an object model every object already has its own identity so why not use that? An object model does not force a need to identify objects by using external values. Using external identifiers strongly couples the rest of the system to choices made in the user interface, an effect to be avoided. In order to decouple the choices made in user dialogues from the rest of the system, the first messages of the main interactions should contain parameters that are references to objects – for example, `checkIn(jill)`, where *jill* is an object of class *Guest*, as opposed to `checkIn("Jill")`, where the identifier is a string.

Remember that we have been using concrete examples to construct interaction diagrams. Inside the class *Hotel*, for example, the above operation may be defined as:

`checkIn(newResident : Guest)`

Clearly, in order to pass an object identifier as a parameter, we need some way for the user interface to obtain the necessary object reference. A common way of doing this is for the interface to know all *Guest* objects, and display them in a list from which the user can make a choice. Even though what is visible in the list are strings representing the guests' names, when a user selects a guest from the list, the value obtained is the reference to a *Guest* object, not the name of an object.

Alternatively, the user might enter the customer's name, swipe their card, or scan the bar code on a voucher. Whatever method is employed, an important design issue is that this process of locating the relevant business objects should take place in the user interface and *not* in the business model. It is then possible to change the way customers are identified without needing to make any alterations to the business model. Because the business model only deals with the business objects, and the identification of the relevant objects is localised in the user interface, only the latter will be affected by an alteration to how identification is performed. The business model is insulated from such changes.

If the interface offers something like the pick list described above, then how will it populate the list? It will need to ask the business model for a list of all *Guest* objects. To satisfy it, we will need additional operations in the business model, such as `getAllGuests()`, that are there solely to support the user interface. This shows that even with the best design decisions, there will be points where user-interface decisions are coupled to the business model. The goal is to minimise the degree of such coupling.

We also need to consider how the business objects will be created. Clearly there must be operations in the business model for this purpose; object creation must be initiated from the user interface, because information about new customers, rooms, reservations

See Section 3 of *Unit 6*.

Different system architectures may place different constraints on where such objects are created and destroyed.

and so on originates there. In contrast to the messages that initiate use cases, object-creation messages *will* include external data such as customer names and reservation numbers.

So, for example, in *Hotel*, we might have an operation such as the following, which creates a new *Guest*.

```
createGuest(lastname : String, firstname : String, address : String)
```

Once business objects have been created using operations similar to this, they can be assigned to variables, or saved in a suitable collection. Subsequently the user interface will use the business objects directly, and operations in the business model will use these objects rather than using *String* objects or other data to find the objects.

3.1 Preconditions

Use cases specify how the system changes by specifying a precondition and a postcondition. The precondition says what can be assumed about the initial state. The postcondition says what must be true afterwards. For example, a *BankAccount* might have a debit operation with a precondition that the amount to be debited is greater than zero, and that the account holds sufficient funds to perform the debit operation without exceeding the overdraft limit.

But whose responsibility is it to ensure that the precondition really is true? Should the operation assume that the precondition is true, since the caller has invoked the method? Should the operation check that the precondition is true, because it does not want to rely on possibly untrustworthy client code outside its control? And what should happen if the precondition is *not* true?

One strategy, called **defensive programming**, requires that preconditions are always checked by the operation itself. Each operation is made responsible for checking whether the precondition is met, and it is safe to proceed – and for taking appropriate action if the precondition is *not* met. The operation will not go ahead if the precondition is false.

With defensive programming, the checking of the precondition is part of the same operation that will also deliver the postcondition, so the relationship between the two may be easier to understand. The main disadvantage of defensive programming is that conditions are repeatedly checked in different places. For example, suppose the debit operation generates a cheque-printing request for the sum involved. The receiver of the request is likely to check all over again that the sum of money is greater than zero. Defensive programming can also create difficulties with *when* state changes are made. For example, if in the bank example, the debit operation checked the request, approved it, and then debited the account, the cheque-printing operation should not verify the balance again.

Design by Contract works somewhat differently from defensive programming. It is the responsibility of the sender of the message to guarantee the precondition of the operation. The operation can therefore assume that the precondition is true, and does not have to worry about either checking it or deciding what to do when the precondition is false. In *Unit 9*, you will see how Design by Contract affects implementation.

The key difference between the two strategies is where the precondition is checked: in defensive programming, the checking of the precondition is done by the operation itself, that is, the receiver of the message; in Design by Contract, the responsibility is shifted to the caller of the operation, that is, the sender of the message.

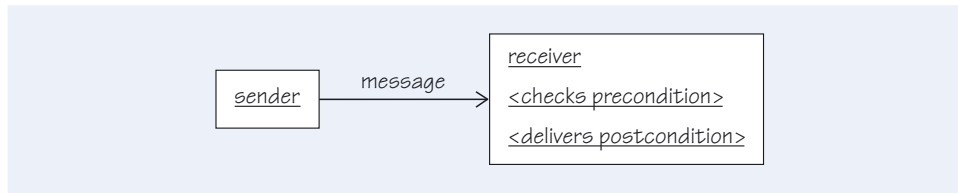


Figure 11 Defensive programming

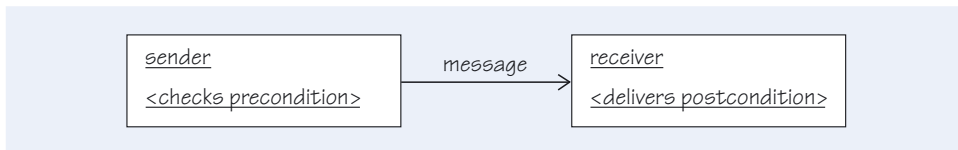


Figure 12 Design by Contract

What should happen in Design by Contract if the sender of a message is faulty and does not check the preconditions as it ought? In that case, the receiver will no longer guarantee the postcondition, because the sender has not honoured its side of the contract. It is difficult to define the failure behaviour, because the condition is not being checked!

Note that the strategies can be combined. The receiver can check certain preconditions that it thinks are critical, and leave others to be checked by the caller.

3.2 Strategies for implementing use cases

You have seen that when you construct interaction diagrams, the first two design decisions that have to be made are the following.

- 1 Which message should be sent from the interface?
- 2 To which object should the interface send the message?

In the check in scenario, we had to consider whether to send a message to the *Hotel* or to the object *jill : Guest*. Hence we were faced with deciding which classes should have the operations that correspond to the identified use cases. There are three different strategies that can be adopted for making this decision, as we now discuss.

One central class

Some designers like to minimise the knowledge the interface must have of the business model by making the interface send all messages to a single object – usually some general object like a *System* or *Hotel*. In the case of the hotel-chain model, each message could initially be sent to the *Hotel*, which would then forward the message to whatever object is to do the work. From the interface's point of view, it has to know the existence of only one object. Each use case will become a method of the *Hotel* object. There is good traceability from use cases to code because every use case links to a method in the same *Hotel* class.

The main disadvantage arises when the one central class becomes overloaded with use cases. One possible solution to this problem is to divide the software system into several packages. Each package could still make use of one class to respond to messages from the interface. The focus of each package would be on the key concepts within each domain, which you will see in the final section of this unit. In the case of the hotel chain, for example, one package could deal with reservations, another with checking in and out, another with payment, and so on.

Actor classes

Other designers are of the opinion that a clearer structure will result if the message is sent to the software object corresponding to the real-world actor who initiated the operation. For example, a person arriving at a hotel initiates checking in. He or she talks to a receptionist, who uses a keyboard to stimulate a user interface to send a message. The current strategy means that the message ought to go to the *Guest* object corresponding to the person trying to check in. Traceability is rather different from that for one central class. To locate the code for a use case, the class corresponding to the initiating actor must be searched. A weakness of this approach is that the class *Guest* includes methods that are related to the role of *Guest* in this particular application, thus reducing its usefulness in another application. There is a further complication when there are two actors that can initiate an interaction, as in the case of making a reservation illustrated in *Unit 3*.

Use cases as classes

In a third strategy, interface messages are sent neither to a system object like a *Hotel*, nor to the object corresponding to an actor, such as a *Guest* or a *Receptionist*. Instead, a new class is defined for each use case. There might be classes such as *CheckerIn*, *CheckerOut* and *ReservationMaker*. Each would have a method with a name like *run*, with a suitable number of arguments. The user interface would then create a single instance of the appropriate class, initialise it suitably, and then send the *run(...)* message. For example, we might send the message *run(jill)* to a *CheckerIn* object, which assumes that we have previously identified the right guest with the aid of another class called *GuestIdentifier*. Component classes such as *GuestIdentifier* are likely to be derived from a detailed use case model.

Not surprisingly, this strategy employs what are known as **use case objects**. Following this strategy is one way to overcome the reuse limitations posed by the strategy of actors as classes. For example, the core concepts or business objects in the hotel domain are the rooms in each hotel and the guests that occupy them. They are unlikely to be different as you move from one hotel chain to another. What are likely to change are the rules and workflows that apply in different hotel chains at different times. One hotel chain may force reservations before allowing anyone to check in; others may have different rules. The same hotel chain may even change its way of working – probably the reason why they want a new software system, in fact! As a result, the user interface is the part of a software system most likely to change. Working with use case objects lets you change or even replace the software to implement a given scenario so as to minimise the effects upon the core concepts. For example, if you changed the means of identifying each guest, the change would affect the implementation of the *GuestIdentifier* class, but not that of any of the business objects. The *CheckerIn* class would be affected only if the interface in the *GuestIdentifier* class had been changed.

Advocates of such use case objects appreciate the traceability they provide from each use case to a class. Each use case class contains information relevant to the use case alone; so each use case can be understood in isolation. If you want to collect management information about the use case, such as frequency of use or timing histories, having an object provides a natural place to put such functionality. When you look at patterns in *Unit 10*, you will see that this is an example of the Command pattern. A possible disadvantage of the approach is the large number of extra classes that must be defined – one for each use case. Another possible disadvantage is that many of the use case classes can be very similar, resulting in duplicated code and more difficult maintenance.

SAQ 4

- (a) What is the main disadvantage of defensive programming?
- (b) Which of the three strategies is being used when we place methods such as `createGuest(lastname : String, firstname : String, address : String)` in the *Hotel* class?
- (c) Are use case objects consistent with the twin aims of high cohesion and low coupling (terms that were introduced in *Unit 1*)?

ANSWER.....

- (a) The main disadvantage of defensive programming is that the constraints within each precondition end up being repeatedly checked in both the client and the supplier.
- (b) We are following the strategy of using one central class here.
- (c) Yes and no. Yes, because high cohesion asks us to ‘do one thing and do it well’. A *CheckerIn* class should only support the one activity of checking in. No, because having an extra class for each use case does introduce more coupling through the extra associations involved. However, the new classes introduce barriers that limit the impact of a change to a software system.

Exercise 2

Suppose you were designing the graphical user interface (GUI) part of a hotel system, and you were concerned with checking in people known to the system, as well as new people unknown to the system. What operations on the business model might be needed to support a usable GUI?

Solution.....

Checking in unknown people would involve the creation of a new *Guest* object. So somewhere in the model, there must be an operation `createGuest(firstName, lastName, address)`.

To check in a known person, we will need to find the business object that corresponds to that person. This mapping might be done by having a pick list of all people known to the system, which would require a business-model operation that returns a collection of guests, for example, `getAllGuests`.



Exercise 3

Recall the library system, which you first saw in *Unit 3*. Suppose you wanted to minimise the dependency of the user interface on the rules and concepts of the library domain. Which messages should be sent from the user interface, and to which object should the user interface send its messages?

Solution.....

A solution can be found from the first of the three strategies that were discussed above. The messages sent from the user interface correspond to each of the library's use cases. Look at Figure 10 of *Unit 3*, from which you can identify 6 messages originating from the user interface:

- ▶ `reserveBook(...);`
- ▶ `issueCopyOfBook(...);`
- ▶ `returnCopyOfBook(...);`



- `updateCatalogue(...)`;
- `enrolNewMember(...)`;
- `lookThroughCatalogue(...)` or, more simply, `browse(...)`.

Each one would be sent to an instance of one central class, typically a general object like a *System* or a *Library*. Although borrowing books and returning books might share common operations, all 6 messages might be needed to maintain the traceability between the requirements and your implementation.



Exercise 4

Assume the business model shown in Figure 13. Redraw the sequence diagram of Figure 12 in *Unit 6* so that use case objects are used, according to the classes and operations shown in Table 1. You may assume that an instance of the use case class *CheckerIn* has already been created before the message *run(s)* is sent.

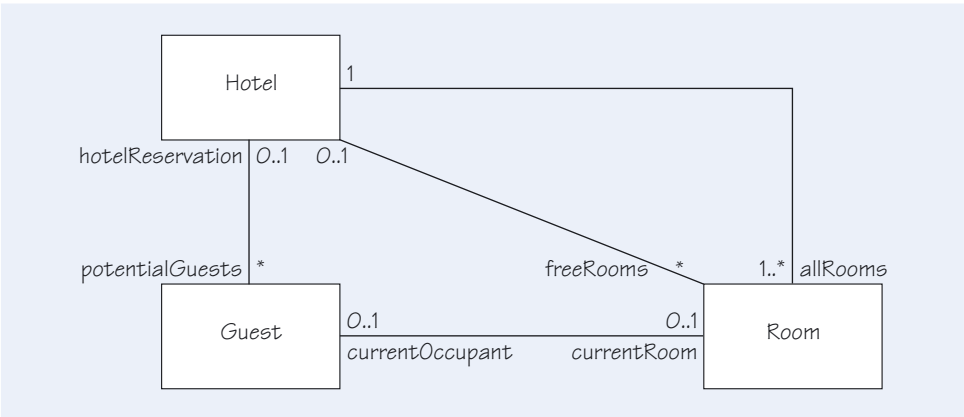


Figure 13 The hotel business model (without the *UserInterface* and *CheckerIn* classes)

Table 1 Operations for the check in use case, using a *CheckerIn* class

Class	Operations
<i>Hotel</i>	<i>identifyGuest(s:String)</i> <i>findAFreeRoom()</i>
<i>Room</i>	<i>accept(g : Guest)</i> <i>setOccupant(g : Guest)</i>
<i>Guest</i>	<i>setRoom(r : Room)</i>
<i>CheckerIn</i>	<i>run(s : String)</i>

Solution
Our solution is shown in Figure 14.

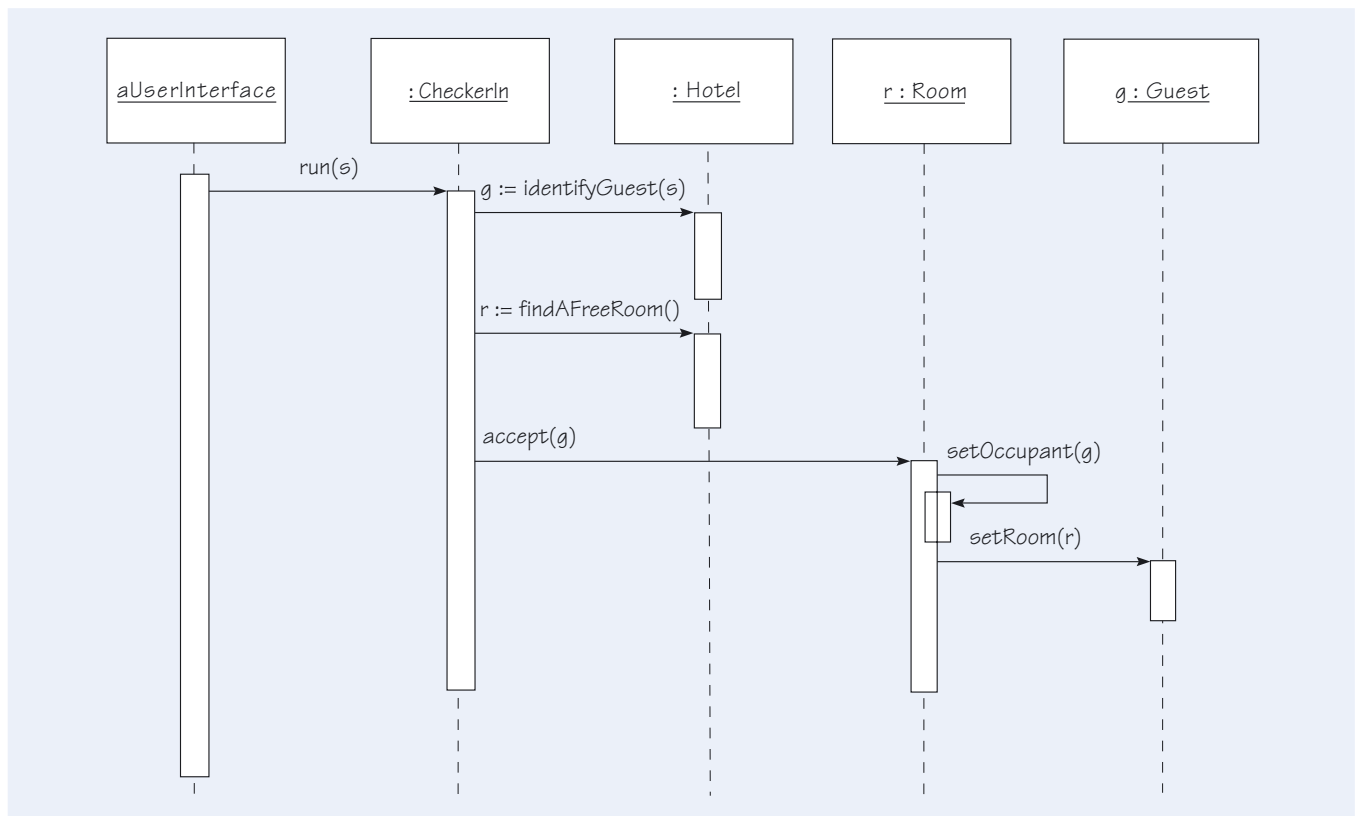


Figure 14 Using a use case object for the *check in* use case

A reference to the current room must be kept in the instance of class *Guest*. There are two options for sending the message *setRoom(r)* to *g* – either from *r : Room* or from *: CheckerIn*. The first option has been used in Figure 14, to minimise the coupling that would otherwise have been introduced by using a *CheckerIn* object, and to follow our earlier advice that one of the classes should be responsible for maintaining consistency of the relationship. See also Section 5 of *Unit 6*.

3.3 Summary of section

You have learnt that there are a variety of design philosophies that you can follow, each one affecting your design in different ways.

- ▶ Other than when invoking object-creation operations, user interfaces should communicate with the business model entirely in terms of business objects, and not via external names or identifiers. The need for the interface to do the mapping from external identifiers to objects may require the business model to support extra queries that are not actually needed by any use case, such as *getAllGuests()*.
- ▶ There are two very different strategies relating to the locations of the operations that enforce preconditions. Design by Contract is generally preferred to defensive programming.
- ▶ There are three strategies for choosing which objects in a software system are to receive messages from the user interface: one central class, actor classes, or use case classes.

4

An introduction to state machines

In this section we will describe a technique for modelling the life history of an object of a class as part of the dynamic behaviour of a system. That is, you will be shown how to model the changes within an object as a result of receiving a message.

In the world that you are familiar with, things happen, and they do so at the most unexpected and possibly inconvenient times. These happenings are known as events, and they can be modelled as part of the behaviour of a system. An event is an occurrence of a phenomenon at a certain moment in time. The occurrence of the event itself is assumed to have no duration. For example, the keyboard strokes that were generated when writing this text are a sequence of events performed by the user of a word-processing software system. At some point, this text was printed as a result of another sequence of events. Up to now in this course, the sending of messages between objects has been the most common kind of event. In this section, you will see a technique for modelling the behaviour of an object over time.

A **state machine** is a model that shows how an object changes from state to state in response to events. A diagram of a state machine is called a **statechart**. It is important to understand the difference between *states* and *events*. A **state** represents some condition or situation in the life of an object. For example, a car's motor might be *running*, or a window might be *shut* or *open*; these are states. A state is not an action, but is instead a description of something about the object that will remain true until something happens. A car's motor might *start* (an event) to put it into the *running* state. A person might *fall*; this is a description of an action, which is not a state. However, we could say that the person is *falling*, a state that will continue until some other event happens (such as landing on the pavement).

A familiar real-life example of a state machine is a retractable ballpoint pen, which has a button that you press to extend the ball point when it is retracted, or to retract it when it is extended. The point has two states, *extended* and *retracted*, as shown in Figure 15. The figure shows the features common to all statechart diagrams: *states* (in this case, two – *extended* and *retracted*); *events* (in this case, only one – *button clicked*); and *transitions* (in this case, one from *retracted* to *extended*, and one from *extended* to *retracted*). Transitions occur as a response to an event, and cause a change of state.

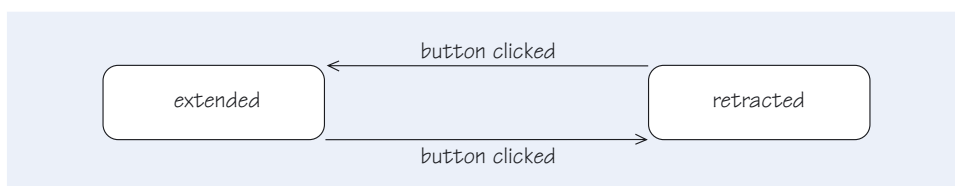


Figure 15 Two simple states for a ballpoint pen

Notice also that the response of the object to the event (what state the object transitions to) depends on what state the object is in – it is state dependent. Figure 16, a description of starting and turning off a car motor, further illustrates the ideas of states, events and transitions. The ignition-key system in most motor cars has (at least) three positions: 'off', 'run' and 'start'. Furthermore, most cars will not permit you to turn the key from the 'run' position to the 'start' position more than once. You must turn the key to the 'off' position before you can turn the key to the 'start' position again. Figure 16 shows the states that the car's motor system can be in. The car starts in the

off state, with nothing running and nothing powered. If the event *key turned to 'run'* occurs, the car enters the state *electricity on motor off 1*: the system has power, but the motor is not running. At this point, the user can either (a) turn the key to the 'off' position and transition back to the *off* state, or (b) turn the key to the 'start' position to attempt to start the motor, transitioning into the *starting* state. After a time, the user will turn the key back to the 'run' position, and either the motor will be running or it will not. If the motor is not running, then the car transitions into the *electricity on motor off 2* state, where the user can leave the key at 'run' (and run down the battery), or turn the key to 'off' to go to the *off* state to try again. If the motor is running, the car transitions to the *motor running* state. The motor will stay running until either the key is turned to the 'off' position or the motor stalls.

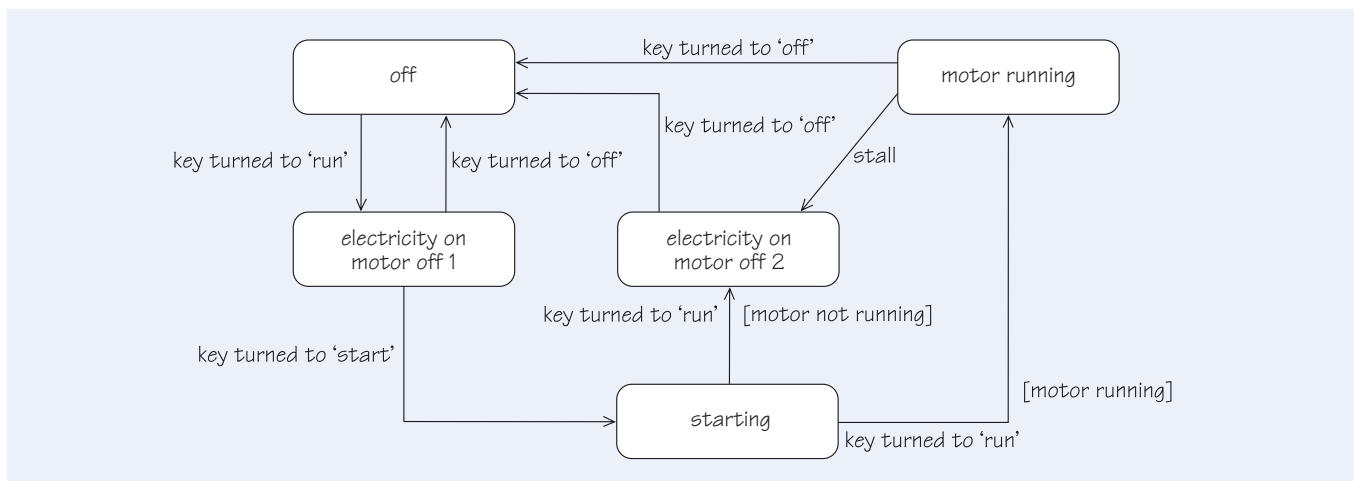


Figure 16 Starting and turning off a car motor

There are two *electricity on motor off* states because we want to model when it is possible to turn the key to start and when it is not. In the first state, it is possible to turn the key to start. In the second state, it is not.

The text in between the brackets [...] is a guard. A guard is used to choose which transition to take if the same event appears on more than one transition, as does the event *key turned to 'run'* when transitioning out of the *starting* state. Guards are discussed further below.

The concept of a state machine is well established in computing. A state machine is a mathematical model of a system that represents the legal internal configurations of the system. Each valid configuration reflects past inputs and determines what the system does on subsequent events. The current internal configuration of a system is its state. Change takes place through transitions from one state to another. Transitions are *fired* or *triggered* by events. Which particular transition is fired depends on the current state. In UML, the term 'state machine' refers to the technique used to model behaviour in terms of states and transitions. A UML statechart diagram is a graphical representation of a state machine showing states, transitions and events. Referring back to Figure 15, the states are *extended* and *retracted*. Each state has one transition leaving it, fired by the event *button clicked*. In Figure 16, the states are *off*, *motor running*, *electricity on motor off 1*, *electricity on motor off 2* and *starting*.

Unlike a sequence or communication diagram, a statechart diagram does not represent interactions between objects. Instead, a state machine models the life history of a single object: its initial state when first created; the different states it can be in while it exists; and how it can move from state to state in response to events – the state transitions that are possible. If the object is destroyed at some point, the state diagram will include an exit state.

Figure 17 presents another example to illustrate how a statechart diagram can represent the behaviour of an object upon receiving a message, and how that behaviour depends on the values of that object's attributes and links. Recall the hotel example used in *Unit 5*, where we considered the problem of room occupancy. A room can either be occupied or unoccupied. Figure 17 shows how a state machine for instances of the class *Room* can be represented in a statechart diagram. A room becomes occupied when a particular guest checks into the hotel. It becomes unoccupied when that guest checks out.

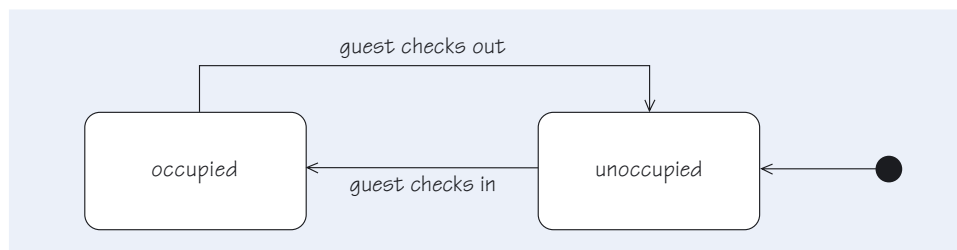


Figure 17 Two simple states for a *Room* object

The two main elements in the statechart diagram in Figure 17 are:

- ▶ the states involved, which are represented by the round-cornered rectangles containing their names;
- ▶ the transitions, which are represented by arrows, and identified by the events that give rise to them.

One question that must be answered is what state an object enters when it is first created. Figure 17 shows which state is entered when a *Room* object is created by the addition of a single transition from a special start marker. In UML, a solid black circle is used to show what is known as the **initial state**. In UML, an initial state is not a true state, such as *occupied* or *unoccupied*, but is instead a way of indicating which of the real states the object will enter first. So in Figure 17, when each *Room* object is created, the values of its attributes and links must be set so that it is in the *unoccupied* state.

The events that fire the transitions have been given simple names – *guest checks out* and *guest checks in* – to indicate which events give rise to the transitions. However, we can do better than this by using information gleaned when constructing interaction diagrams. For example, if you look at Figure 18, you will see that there is an exchange of messages between *Hotel*, *Room* and *Guest* that results in a particular guest occupying a particular room. In other words, this exchange of messages results in a change of state of the room.

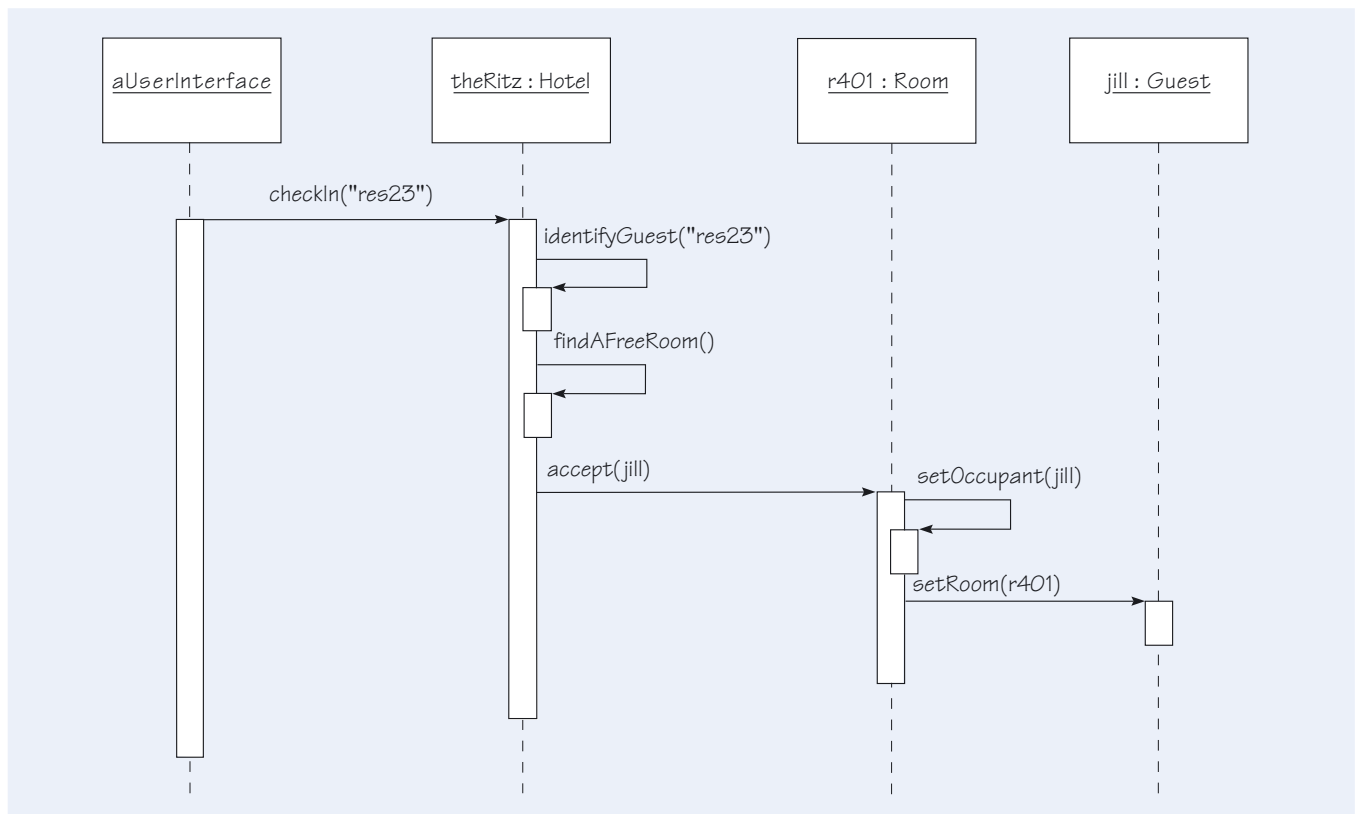


Figure 18 One way to check a guest in to a room

The receipt of a message is always an event. It might result in a change of state of the receiver object, or it might not: it could simply return some information, but not change anything. A message that causes a change of state is called an **action**.

An event, when it occurs, has consequences. One kind of consequence is a change of state. Other consequences are recorded in the action part of the label on the transition. That is, instead of labelling a transition just with a simple name, we also record the actions (consequences) that result from the occurrence of the event.

Figure 19 shows a revised version of the statechart diagram of Figure 17; you might have created such a diagram after constructing the interaction diagram in Figure 18. In the checking in process, a *Room* object is sent the message *accept(aGuest)*. This is an event that causes the *Room* object to respond by sending itself the message *setOccupant(aGuest)*, an action. The result is that the *Room* object changes state from *unoccupied* to *occupied*. So you can see that transitions from state to state are not the only expressions of behaviour in a state machine.

We have used the *aGuest* to indicate that, in general, the argument will be an object from the class *Guest*.

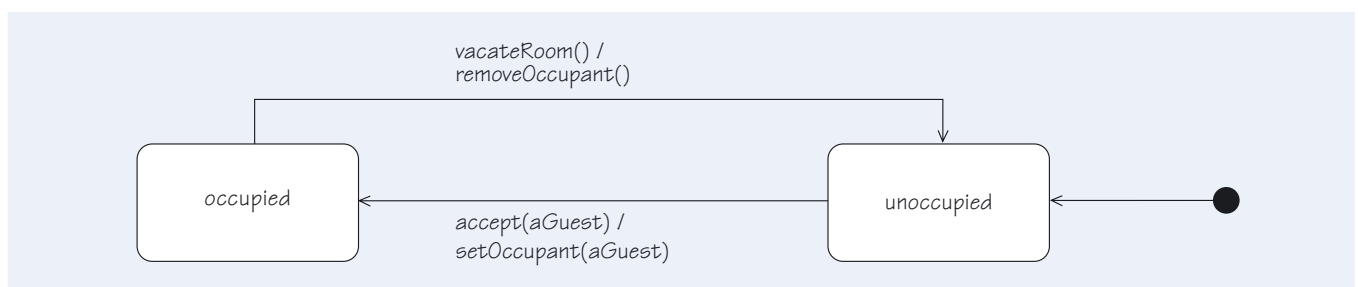


Figure 19 Adding detail to a state machine for a *Room* object

Figure 19 illustrates the notation for labelling transitions in UML: *event / action*.

Most CASE tools will automatically generate the slash used to separate events from actions. However, each one is likely to have different ways of dealing with action sequences.

A single event may often lead to a sequence of actions. In the check in example, there are two actions taken by each *Room* object. So the transition from *unoccupied* to *occupied* in Figure 19 should really be labelled as follows.

accept(aGuest) / setOccupant(aGuest); aGuest.setRoom(self)

The second action, *aGuest.setRoom(self)*, is a message sent to another object. In this case, it is the one identified by the reference passed to the *Room* object from the message *accept(aGuest)*. The association between *Guest* and *Room*, which you saw in Figure 8 of *Unit 5*, is navigable in both directions. So we need a pair of messages to confirm that a guest has occupied a room.

If there were more messages to be sent, they could be added to the action sequence following the event. Each action is separated from the next in the list by a semicolon. The order in which the individual actions take place is defined by the order in which they appear in the list. No matter how many actions there are in response to an event, the object cannot enter the target state until the last one has completed. In Figure 19, for example, the *Room* object does not enter the *occupied* state until the action *setOccupant(aGuest)* has completed. In the modified version, the room object does not enter the *occupied* state until the action *aGuest.setRoom(self)* has completed.

SAQ 5

- Why might two objects of the same class respond differently to the same message?
- What is the most common form of event that causes a transition between two states? How is it shown in a statechart diagram?
- What is the difference between an event and an action?
- What is the main constraint on the kinds of action that may be shown in a state diagram?
- What is an action sequence?

ANSWER.....

- An object's behaviour will in general be affected by the values of its attributes, which are part of the object's state. If two objects have different values for the same attributes, they are in different states, and therefore they might respond differently to the same message.
- The receipt of a message is the most common form of event that causes a transition between states. An event is used to label the transitions between the states. In a statechart diagram, a transition is labelled with the name of the relevant message, which includes any arguments for that message.
- An event is something done to the object, such as sending it a message. An action is something that the object does, such as sending a message to itself or to another object. An action is an object's reaction to an event.
- Actions should only refer to things that the object 'knows' about. For example, they can refer to attributes, operations and links of the object, as well as to the parameters of the message that caused the transition. An action cannot refer to the state or attributes of another object unless there is some way for these other attributes to be known in a short time without requiring any state changes.
- An action sequence is an ordered series of individual actions that are associated with a particular event. They are written as a list separated by semicolons, and are performed sequentially in left-to-right order. Like actions, action sequences are atomic.

4.1 Guards

Sometimes an event might take place, but you only want a transition to fire under certain conditions. As you have seen in *Unit 3* and earlier in this unit, UML provides the concept of a *guard* to deal with conditional behaviour. In a state machine, a guard is evaluated when an event occurs. The transition, and any actions associated with that event, will only take place if the guard evaluates to true. For example, if we were following a defensive programming strategy, the object itself would want to ensure that guests could check into rooms only if they had existing reservations, rather than depending on the sender to have made the checks. In this case, we could rewrite the *accept(aGuest)* transition in Figure 19 using the square bracket notation for denoting conditions as follows.

```
accept(aGuest) [reservation exists] / setOccupant(aGuest); currentGuest.setRoom(self)
```

If no reservation were to be found for the guest, the actions would not take place, and the *Room* object would not move from the *unoccupied* to the *occupied* state. Note that a guard condition is a form of query about the system and its state: it must not change any values, and it must not have any side effects.

Sometimes the same event can give rise to different transitions and actions depending on the prevailing conditions. Such situations are common where loops are formed – for example, keeping track of how many rooms are available in a hotel, or the number of copies of a video on loan. We can illustrate this by considering what happens when a hotel becomes full. If we identify two simple states *full* and *not full* for the class *Hotel*, what happens as guests check in and out? Figure 20 illustrates this situation. The hotel has a limit to the number of rooms that it can let to guests. At some point, the hotel may find that it cannot accept any more guests. Figure 20 shows that guests can continue to check in until that limit is reached. The hotel is not full provided that there is at least one room that can be occupied. Checking out is not a problem in this simple example. Both instances of the event *checkOut(aGuest)* lead to the *not full* state. However, an occurrence of the event *checkIn(aGuest)* will lead to *either* the *not full* state *or* the *full* state, depending on whether or not the *Hotel* has more than one room unoccupied. Therefore the guard conditions in Figure 20 are mutually exclusive: *[last room]* or *[not last room]*.

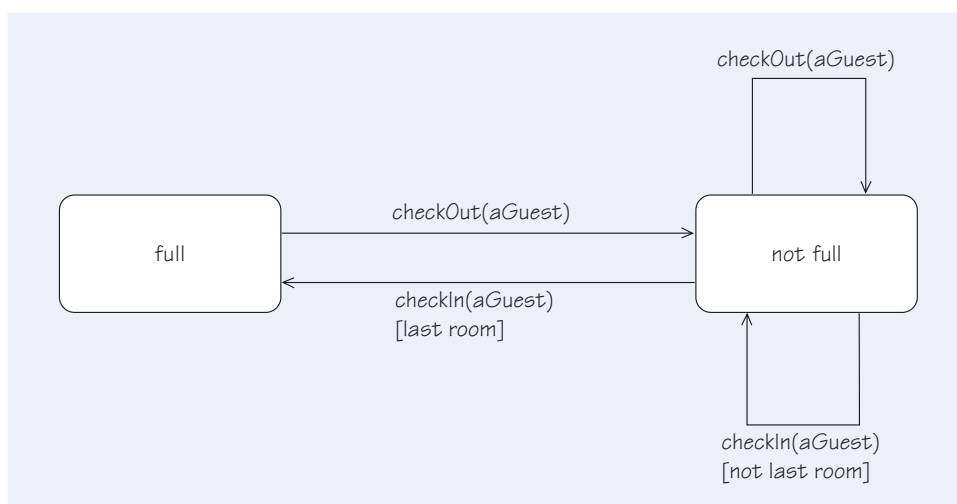


Figure 20 A simple state machine for the *Hotel* class

Figure 20 contains two instances of a special kind of transition, known as a **self-transition**, which originates in one state and returns to the same state. They are commonly found in state machines that model loops or cycles.

A self-transition can also be represented as a curved arc as long as it starts and ends in the same state.

From the perspective of checking in and checking out, the state machine in Figure 20 is incomplete because it does not deal with the *empty* state. If every guest in the hotel checks out, the hotel is empty. Figure 21 shows one way to follow the checking in and out of guests into a hotel that might be full, empty or somewhere in between. The maximum number of rooms in a hotel is denoted by *hotelLimit*. A simple counter, called *availableRooms*, keeps track of the number of unoccupied rooms in the hotel.

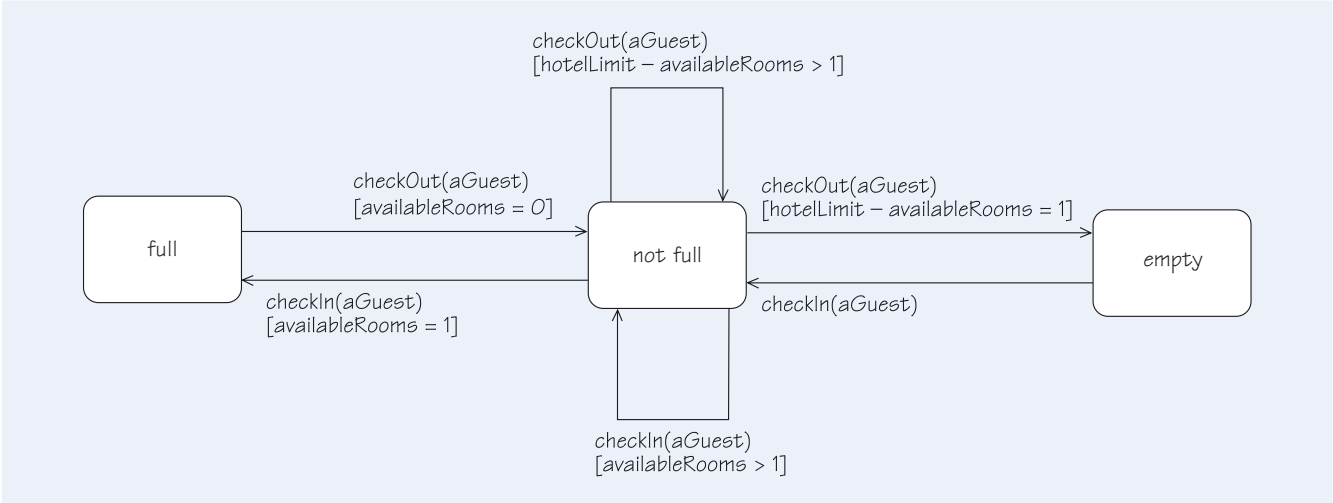


Figure 21 A full hotel or an empty hotel?

If the same event is used on more than one transition, you must make sure that no more than one guard can evaluate to true. UML says that only one transaction will fire no matter how many guards are active, but it does not say which one. To avoid any possibility of ambiguity, just one transition should be able to fire in any given situation. For example, which transition will fire if the value of *hotelLimit* is 100 and the value of *availableRooms* is 2 when the event *checkIn(aGuest)* occurs, and the *Hotel* object is in the *not full* state? First of all, we do not need to worry about the *hotelLimit* under these conditions. There are two possibilities: either the *Hotel* object moves to the *full* state, or it remains in the *not full* state. Since the value of *availableRooms* is greater than 1, it is the self-transition that is triggered.

We recommend that you provide a guard on each event as illustrated in Figure 21. Doing so enables you to check that when the same event occurs more than once – for example, *checkout(aGuest)* occurs three times in Figure 21 – the associated guards are mutually exclusive (only one can be true at any one time).

In a real hotel chain, there are likely to be a number of complications, and you would need to revise your state machine accordingly.

SAQ 6

- (a) What is a guard, and how does it protect a transition?
- (b) Look at Figure 20, which only considers whether or not an object of class *Hotel* is full. Why are there no guards on the transitions for the *checkOut(aGuest)* event?
- (c) Why were guards introduced on the transitions for the *checkout(aGuest)* event in Figure 21?

ANSWER.....

- (a) A guard is a Boolean condition that is applied to a transition; the guard must be either true or false. A guarded transition can only take place when the specified guard is true.
- (b) A guest who is checking out cannot make the hotel full, so no guard is needed for the *checkOut(aGuest)* event.

- (c) The example in Figure 20 is incomplete. Figure 21 deals with both extremes – when the hotel is full and when it is empty – as well as the possible configurations between full and empty. This results in *checkout(aGuest)* occurring in three transitions, two of which (from *not full* to *empty* and from *not full* to *not full*) require guards to ensure mutual exclusion between them. For completeness, we added a guard to the remaining *checkout(aGuest)* event (from *full* to *not full*) as a check on the mutual exclusion between all three uses of *checkOut(aGuest)*.

4.2 Exceptions

There are two other possible transitions that are not shown in Figure 21. The state machine does not show what happens when:

- ▶ the event *checkIn(aGuest)* occurs and the *Hotel* object is in the *full* state;
- ▶ the event *checkOut(aGuest)* occurs and the *Hotel* object is in the *empty* state.

You could view the diagram in Figure 21 as showing the normal processing, omitting the exceptional cases. Certainly we would not expect the *checkOut(aGuest)* event to occur if the *Hotel* were empty; if it did, we would consider it an error. Similarly, if the *checkIn(aGuest)* event occurred when the *Hotel* was full, this would be another error, given that an instance of the class *Guest* should not be able to check in without a reservation. However, if it were decided that the hotel system should cater for people requesting a room without having a reservation, the *checkIn(aGuest)* event occurring in the full state would undoubtedly occur in real life, and should be captured as part of normal processing.

The question for us here is how best to deal with error situations. One useful guideline is to model only what happens if the error is in some sense *expected*, and a special response to the error is needed. For example, you might want the user interface to ask the receptionist if the person was in fact checking out if someone tries to check in to a full hotel.

SAQ 7

- (a) Figure 22 shows a simple statechart diagram for the class *Copy*. What should happen if the *copy returned* message arrives when an object of the class *Copy* is in the state *on shelf* (assuming that we are not using Design by Contract)?

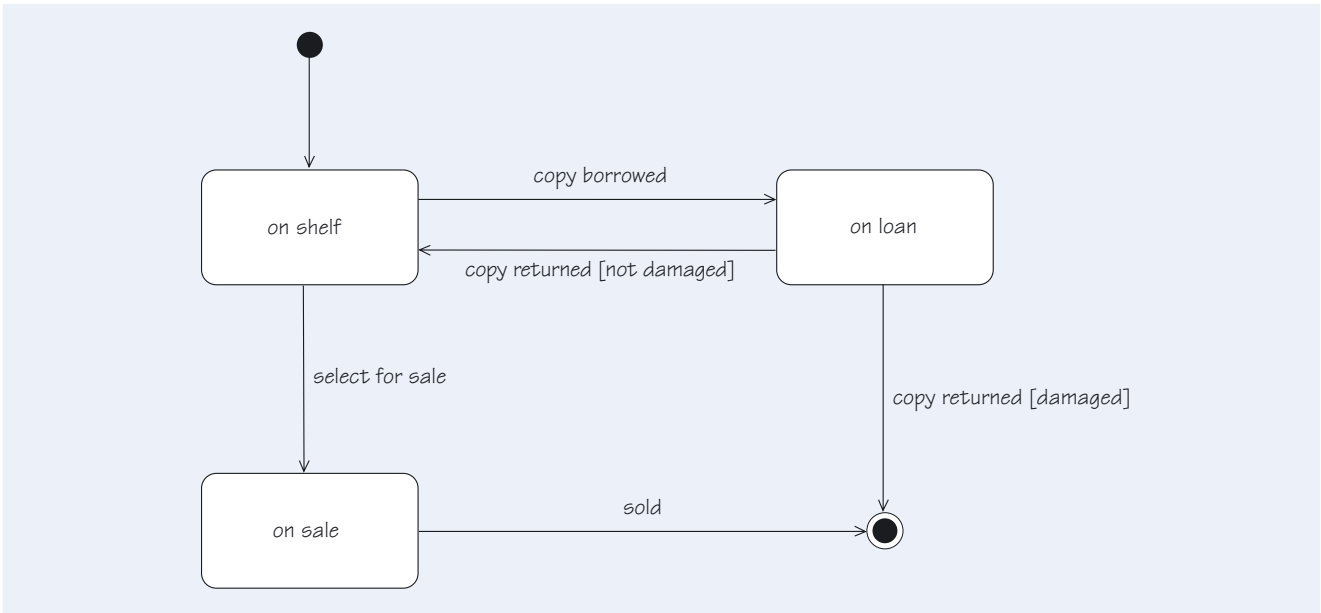


Figure 22 The life history of a *Copy* object

- (b) Give an example of an architectural decision that would provide a general solution to the problem of unexpected messages.

ANSWER.....

- (a) Something has gone wrong. The state of the system does not describe the fact that the copy of the book is not on loan. There must be some means of indicating or recording an error, such as a message to a log file.
- (b) At an architectural level, we might introduce a single object of a class *Error*, which is globally accessible to objects in the software system. Such an object would be responsible for reporting errors due to unexpected messages, for example.

4.3 Entry and exit events and actions

In UML, activity diagrams and statechart diagrams are closely related. They have a number of common model elements, such as the use of an initial state, signified by a black circle. However, there are some important differences, most notably between actions and activities. An *action* is an executable atomic computation that results in a change of state in a model or, more generally, leads to the return of a value. Since an action is atomic, it cannot be interrupted by an event, so an action is expected to run to completion. In contrast, an **activity** is a substructure, usually a sub-state machine consisting of a sequence of states that may be interrupted by other events. Because an activity is itself a state machine, it can lead to a number of actions, depending upon the prevailing conditions.

The notion of activities implies that states may be nested within other states, which is true. A single state can have a substructure that relates to an activity of interest to you as a developer. UML provides ways of showing an activity on a statechart diagram, which avoids or defers the need to add complexity to your state machine by revealing all its substructure. However, it may be necessary to show that one or more states do contain important activities. You will see this in the next section.

You may find that all transitions into a state share some actions. You can avoid putting these actions on every transition by using two special actions for state machines in UML:

- ▶ an *entry* event, which is triggered every time an object enters a state;
- ▶ an *exit* event, which is triggered every time an object leaves a state.

The use of entry and exit events and their associated actions allows you to reduce the complexity of some state machines. Instead of writing the same action on two or more transitions leading to or from the same state, that action can be associated with the state's entry or exit event. For example, Figure 19 can be redrawn by adding an action to both an entry and an exit event, as shown in Figure 23. The two statechart diagrams are equivalent.

Entry and exit events are not allowed to have guards.

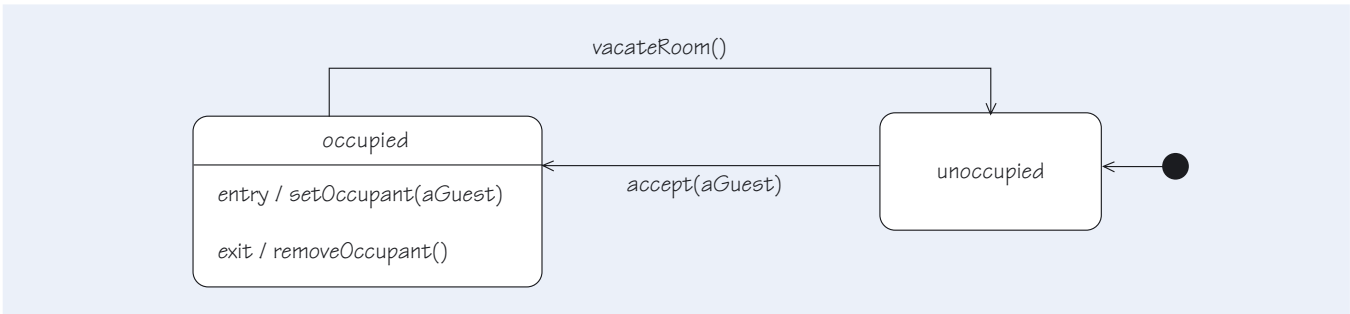


Figure 23 Using entry and exit events for a *Room* object

Whenever you use an entry event, you must remember that all transitions into the state will cause that entry event to occur. This means that the actions associated with the entry event must be appropriate for *all* incoming transitions. Similarly, the actions on an exit event must be appropriate for *all* outgoing transitions. In the case of a self-transition, both the entry and the exit events will occur, so you must take care to ensure that their actions are consistent with each other. While your chosen entry and exit events might be fine individually for non-self-transitions, they may be inappropriate for self-transitions.

UML also provides a way of dealing with the situation in which an event should cause an object to return again to the same state but without performing the entry and exit events. An **internal transition** is a special transition that does not involve a change of state. In contrast to a self-transition, an internal transition does not cause the execution of any entry or exit actions. An internal transition is typically used to handle an interrupt where you want some action to take place that does not affect the current state. Like entry and exit events, internal transitions are written inside the state to which they refer. Use a suitable event name followed by the action that follows it. For example, when a hotel's upper limit to the number of available rooms changes, the number of available rooms should be revised. Figure 24 shows how we might use an internal event to achieve the change in the *not full* state. In effect, a *Hotel* object receives the message *hotelLimitChanged()* – an event – and responds by sending the message *resetAvailableRooms()* to itself – an action.

The details of such a scenario are not relevant here. You would, however, be advised to model such a requirement with one or more interaction diagrams. For example, *hotelLimitChanged()* might just signal a change in the value for each upper limit or you might prefer to pass in the new value as an argument to the method.



Figure 24 An internal transition within a *Hotel* object

SAQ 8

- (a) What is an entry event, and how does it contribute to the maintenance of a state diagram?
- (b) What is the benefit of using an internal event as opposed to a self-transition?

ANSWER.....

- (a) An entry event can be used where there are multiple transitions, with the same actions, leading to a particular state in a given diagram. An entry event occurs every time an object enters the state that it annotates. Entry events reduce the risk of introducing errors, because the action sequence is written once (associated with the entry event of the state) rather than many times (on each of the transitions leading to that state).

- (b) When there are entry and exit events that might interfere with a self-transition, an internal event is useful, as the entry and exit events are not triggered.

In the case of nested states, as you will see in the next section, initial and final states have other restrictions.

4.4 The final state

We have shown that a state machine starts with a single initial state, but what do we use to show that processing has completed or that an object's lifecycle has ended? UML uses a **final state**, signified by a black circle surrounding a solid black circle, to show that processing has been terminated. A final state may be active for a period of time, unlike an initial state that results in an immediate transition to a successor state, such as the *unoccupied* state in Figure 17. An initial state has only one outgoing transition, whereas a final state can have many incoming transitions but no outgoing transitions. As a consequence, a statechart diagram shows no more than one initial state, but there can be zero, one or more final states.

In a simple state machine, initial and final states signify the creation and destruction of an object. Suppose, for example, you were asked to model the behaviour of the books in a lending library. Newly purchased copies of a book are put on the shelf for lending to the library's members. Damaged copies are removed from stock. At certain times of the year, each library is allowed to select copies of books to go on sale to the public. Figure 22 above shows a simple state machine that follows the life history of each instance of a *Copy* object; you might draw such a diagram during your analysis of the problem. There are two possible transitions that lead to the final state for each *Copy* object. Once a physical copy of a book has been removed from the library's shelves, the corresponding *Copy* object can be destroyed.

SAQ 9

- (a) What does a final state signify in a state machine?
- (b) In what ways does a final state differ from an initial state?

ANSWER.....

- (a) A final state is used to show the point or points where the object in question has finished processing or has reached the end of its life and will be destroyed. Although we might show more than one final state in a statechart diagram, each copy represents the same final state. (In the next subsection, you will see that a stop marker can be used to indicate that a composite state has finished executing.)
- (b) There can be zero, one or more final states but at most one initial state. A final state can be active for a period of time, whereas an initial state results in an immediate transition to a successor state. A final state can have several incoming transitions and no outgoing transitions, but an initial state has no incoming transitions and only one outgoing transition.

4.5 When to use state machines

As with any other kind of model, you should try to design your system so that each state machine for a class is as simple as possible. As a state machine becomes more complex, it becomes harder for client objects of other classes to use it correctly. For example, if a *Copy* object in Figure 22 is in the state *on loan*, is it an error for it to receive a *copy borrowed* message? How many extra messages would a client object have to send in order to be confident about sending the intended message *and* receiving some useful result? With a small number of states and transitions, it may be easy to gain some confidence about the state of the intended recipient of a message. As the number of states and transitions increases, it requires more effort to determine the state of the intended recipient of a message. At the same time, you will need to deal with conditional behaviour in the methods for the class in question. As the number of potential pathways through a state machine grows, the more conditional statements you will need to code and test, which makes it harder to verify the behaviour with respect to the software specification.

See *Unit 11* on the subject of testing.

State machines have several benefits.

- ▶ They are a means of elaborating the potential operations within a class – identifying which messages affect which attributes, and considering the consequences for each object belonging to a given class. A state machine can help the developer understand the behaviour of a particular object over one or more use cases. As you saw above, for example, you might model the possible states of a *Room* object in a hotel system for the checking in and checking out of guests. Then, at some later point, you may have to deal with the cleaning and maintenance of each room, which would add complexity to the simple state machine shown in Figure 17.
- ▶ They help you understand the behaviour of an object over its lifetime. The more often an attribute of an object changes, the more likely you are to want to model the object's behaviour in a state machine. If more than one attribute changes often, the need to model the object's behaviour by a statechart increases accordingly. For example, the system may be asked to monitor the fact that a room must be regularly cleaned. Thus *clean* or *dirty* could be added to the list of states for a room. However, an increase in the number of states increases the potential complexity of the behaviour of the object because of interactions between the states. For example, the hotel management may not want a room to be cleaned while it is occupied.
- ▶ They help you understand how an object or a system must respond to events. They, like communication and sequence diagrams, help you understand object interactions. In practice, you should expect to use a combination of techniques.
- ▶ They are a means of ensuring correctness. In a formal development process, you may be required to produce a state machine for each class as part of a given requirement to ensure a level of correctness for the behaviour of the objects within a software system.

At this level of detailed design, it is sometimes easier to implement a class rather than produce more and more detailed diagrams. A state machine is useful when it helps you to understand what is going on and to verify that understanding with the domain experts.

We have now reached a point in the development of a software system where we are very close to producing the code in our chosen programming language.

SAQ 10

- (a) Identify three problems associated with complex state diagrams that might arise when designing classes.
- (b) Suppose the class *Copy* included the attributes *returnDate*, *libraryNumber* and *classification*. Which of those attributes are significant for a state machine for the class *Copy* that has two states called *on shelf* and *on loan*? Explain your choice.
- (c) Suppose you implemented your design for a class without using a state diagram. Are there any subsequent activities where a state machine might help you as a developer? Briefly explain your answer.

ANSWER.....

- (a) Classes with complex state machines can cause three kinds of problem. First, it is harder to write the eventual code for such classes, since there are likely to be many conditional tests to identify the actual state. Second, it is harder to test the class because of the number of choices of pathway through the conditional tests. Third, it is much harder for external code to use a class correctly without some means of ascertaining the actual state of objects belonging to complex classes.
- (b) State machines help model what might happen when a particular object receives a given message, since the behaviour of an object is influenced by the values of its attributes. A state machine tells you about the life history of an object. Therefore you can use the frequency of change of an attribute during an object's lifetime as a means of choosing which attributes are significant when constructing a state machine. In the class *Copy*, for example, the attribute *returnDate* is likely to change often, as it reflects the transition between the states *on shelf* and *on loan*. However, attributes such as *libraryNumber* and *classification* will only change as a result of some reorganisation within the library (probably set during the creation of instances of the class *Copy*). The borrowing and returning of each copy by the library's members do not affect them. Among other things, you might consider an additional attribute that records the actual date when a copy is returned (*actualReturnDate*, say) if you needed to investigate the notion of fines for overdue books.
- (c) You might need to develop state machines retrospectively. For example, you might benefit by preparing a state machine when testing individual classes, to demonstrate that the behaviour of an object over its lifetime satisfies the relevant requirements for its class – usually for the purposes of verification. In addition, a proposed change to a software system might introduce a need to prepare one or more statechart diagrams to show how an object's state would be affected.



Exercise 5

It is possible to have both entry and exit actions on a single state. Suggest a potential benefit to the consistency of your implementation as a result of using entry and exit actions in pairs on the states of an object.

Solution

Entry and exit actions cannot be bypassed, and therefore can be used to tidy up the states within an object and maintain a level of consistency. An entry action can be used to perform the setting up required before the object enters a state. Similarly, an exit action can be used to 'clean up' afterwards.

Exercise 6



Suppose you were developing software for a telephone system. When looking for the objects with ‘interesting behaviour’, you selected the telephone handset, since it represents the principal interface for its users. There are two basic states to consider: *idle* and *active*. They reflect the basic activities of making a call to someone. Either you have picked up the handset and are ready to dial, or you have completed your call. Draw a simple statechart diagram that illustrates this behaviour, giving events and actions.

Solution

Figure 25 shows the transitions between *idle* and *active* for a telephone:

event: *handset lifted*; action: *sound dial tone*

event: *handset replaced*; action: *disconnect*

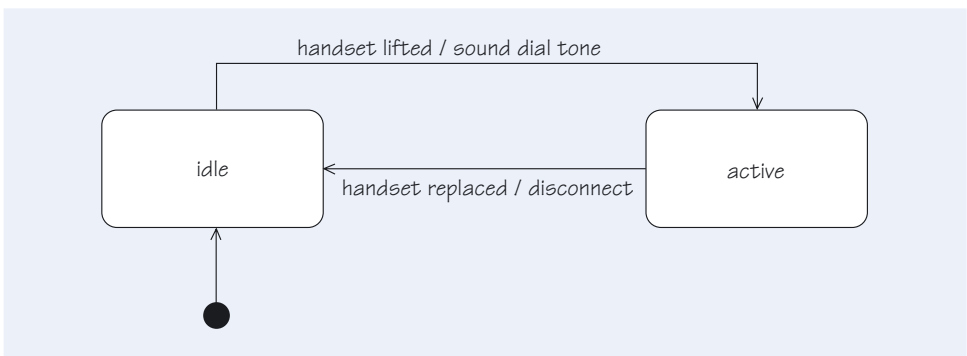


Figure 25 Two basic states for a telephone

Having modelled the simple fact that you are either using a telephone or you are not, your next model might consider the behaviour required to reach another telephone in order to communicate with another person. For example, you need to dial the appropriate number and find out whether the number is busy. In Section 5, you will see that it is possible to model this behaviour by looking at states within states.

4.6 Summary of section

In this section, you have seen how to use state machines to model the dynamic behaviour of an object or a system. Because state machines are modelling the behaviour of an object, they will usually be involved in the behaviour of more than one use case as follows.

- ▶ A state machine in UML contains a number of states that are connected by transitions. The transitions between states are caused by events that may give rise to a number of actions. Initial and final states are helpful when recording the full life history of an object. The transition from an initial state leads to the particular state that an object enters after its creation or initialisation. A final state is reached when an object is about to be destroyed or otherwise stops responding to events.
- ▶ In a simple statechart diagram, the states are shown as boxes with rounded corners, and the transitions between them are shown as arrows. Each transition may be labelled with an event and an associated action. However, a transition from an initial state should not have an event label. In contrast, there may be more than one transition leading to an object’s final state, each one showing the variety of events that take that object to its final state.

- ▶ A transition can be associated with a sequence of actions in UML. Actions or action sequences can also be written inside a state to avoid the firing of any entry or exit actions that apply to that state.
- ▶ State machines are a means of elaborating the operations of a class. It is possible to identify the messages that affect an attribute of that class and the actions that arise as a consequence.
- ▶ Whenever a specific event can lead to different transitions, it is essential to identify the conditions under which each transition occurs. This is achieved in a statechart diagram by the use of guards.

As a result, you can use state machines to identify what is needed to implement an operation of a class.

5

Variation of events and actions

In Section 4, we introduced a technique that is used for modelling the changes of state of an object. In particular, you saw the notation used in the statechart diagrams of UML. In this section, we will introduce further notation from UML, so you can include events other than simple messages, and so you can use composite states, which allow you to model the detailed structure within a single state.

5.1

Other kinds of event

In UML, an event is a noteworthy occurrence that has a location in time and space; it occurs at a point in time and does not have a duration. It is useful to model an event if its occurrence has one or more consequences.

Events are classified as either external or internal. **External events** are those that involve a system and its actors, such as the pressing of a button, the use of a keyboard, or a signal from a sensor. In contrast, **internal events** are those that involve the objects within a system, such as the exceptions used in the Java programming language. In addition, actions are either synchronous or asynchronous, terms that you encountered earlier in this unit.

UML defines four different types of event, which are described in Table 2.

Table 2 UML event types

Event type	Description
Call event	The receipt of a request to invoke a specific operation synchronously. Message passing among objects is the most common form of call event.
Signal event	The receipt of a particular asynchronous signal, such as the user interface detecting a mouse click, which is used to trigger a reaction in the receiver, but with no reply from the receiver.
Change event	An event that occurs when a Boolean expression becomes true after a change in value of one or more attributes or associations. For example, we may need to check <i>when</i> the depth of liquid in a tank exceeds an upper limit. Conceptually, the Boolean expression is evaluated continuously until it becomes true, whereas a guard is only evaluated upon detection of its associated event(s).
Time event	The expiry of a specific deadline. The expression defining the deadline can be absolute, such as tonight at midnight, or relative, the passage of an amount of time since entering the current state.

Generally, call events are synchronous and represent the invocation of an operation. Signal, change and time events are, in contrast, asynchronous. When an object invokes an operation on another, control shifts from the sending object to the receiving one. When that operation is completed, control passes back to the sender. For a signal event, the sending object does not give up control after sending the signal, and the receiving object will respond to that signal (that is, the sender and the receiver are concurrent). Call and signal events have something in common. Both of them involve at least two objects: a sender and a receiver.

Figure 26 shows a statechart diagram for a telephone with two states: *idle* and *active*. You can use a time event, as a self-transition, to sound a warning message if the caller has not begun dialling after a certain period of time. The state of another object, say *localClock*, must be checked after some specified interval. We might also want to sound a warning message when the caller has run out of credit on a phone card, or to prompt the caller to add more money (that is, a conversation will be interrupted).

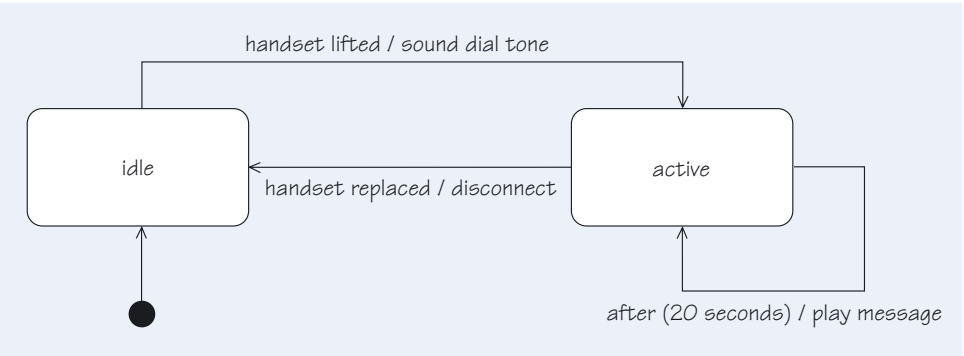


Figure 26 A simple telephone statechart diagram

Signals are common in event-driven systems, and there can be several types of signal that are related by generalisation. Therefore you might model a hierarchy of signal events in order to follow their influence on the flow of control within the software system. Consider the case of a robot in a manufacturing system, where we might model a hierarchy of internal and external signals. External events such as collisions will be important – you do not want robots to run into anyone! Equally, you must be aware of other events that give rise to problems, such as a lack of motive power because of a flat battery. Figure 27 shows a simple signal hierarchy for a manufacturing robot.

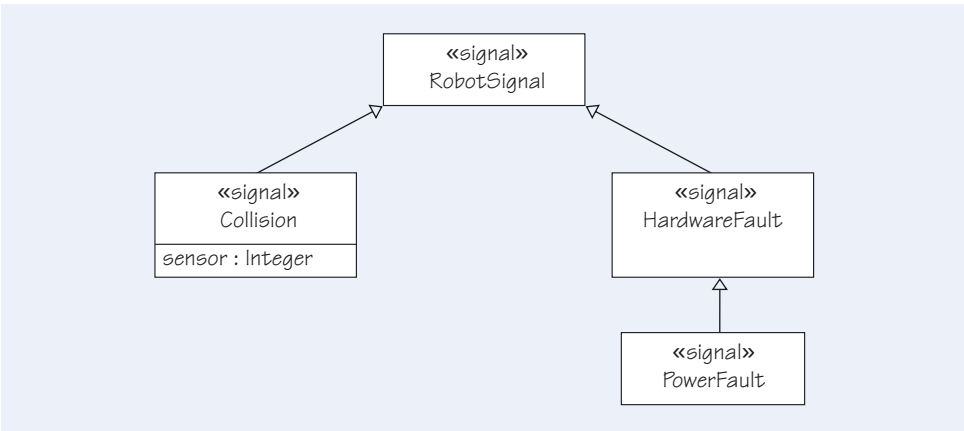


Figure 27 A signal hierarchy for a manufacturing robot

Note how the stereotype «*signal*» is used in the specification of a signal hierarchy.

Signals can have attributes but no operations. You can use the attributes of a signal as the parameters for the signal event. For example, *Collision*(3) might be used to signal that sensor 3 has detected a collision.

Events may interrupt or be detected by polling. A call event is synchronous, whereas an interrupt is asynchronous. Time events will usually result from a clock interrupt (but the clock may be polled), and will most likely be delivered asynchronously. Change events may be declared as significant events, and as such would also be delivered asynchronously. For example, a time event relies upon a check against a clock to evaluate the time expression for that event, the result of which may influence the subsequent flow of control.

Action and activity

In general, objects are passive. Sometimes, however, they adopt an active role, and you may need to model one or more activities within a state machine. An object may be expected to do some work while it is in a certain state. An example of a domain where state machines can help you understand such intricacies is *process control*. One way to understand a complex system is to use a simpler one that illustrates a common feature of interest. For example, you might compare a simple room heater with a large-scale chemical process that needs to maintain a constant temperature. Figure 28 shows a simple state model to represent the use of a room heater. Such heaters usually use a thermostat to set a desired temperature. The heater is turned on or off according to the difference between the current and desired temperatures. You could use a similar model to represent the need to cool a room. Once you have gained an understanding of a simple system, you can apply these principles to a larger system – for example, the need to keep a vat of chemicals at a specific temperature.

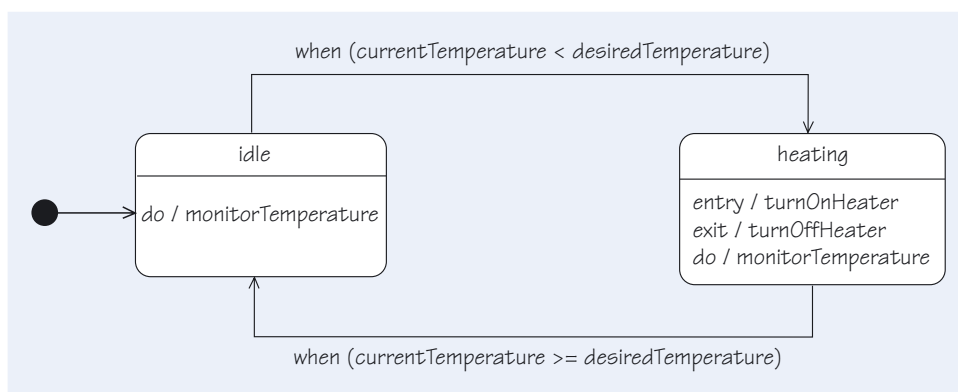


Figure 28 A simple heater state machine

Figure 28 shows two change events to control the transitions between the *idle* and *heating* states. In both states, there is a need to monitor the current room temperature. UML provides the special *do* transition to identify what activity should be performed after any entry actions have been dispatched. That activity can take place for as long as the object remains in the current state, or it can run to completion. Figure 28 shows the same monitoring activity applied to both of the heater's states.

Remember that the main reason for constructing a state machine is to show the response of an object to external events rather than to show the internal processing that follows such events. As Figure 28 shows, such models can help provide a useful view of a problem, which you can use during analysis. However, an activity, such as *monitorTemperature*, is likely to be decomposed during design.

SAQ 11

- In Section 4 you saw examples of message passing in state diagrams. In UML, what kind of event is a message?
- Is a signal a class? Briefly explain your answer.

- (c) What is the difference between a time event and a change event?
- (d) What kinds of event are *entry* and *exit* events?

ANSWER.....

- (a) A message is the commonest case of a call event.
- (b) A signal is a special kind of class. However, there are no operations associated with a signal, and its parameters are recorded as attributes. Because they are classes, signals may be related through an inheritance hierarchy, but they must be separated from the other classes in the software system.
- (c) A time event relates to a specific point in time or an interval. A time event is a Boolean test of a time expression, such as the length of time that must elapse between a named event and a subsequent action. A change event takes place when a test condition evaluates to true at some point in time. A change event is not restricted to time; rather, the transition can fire when some condition is true, such as the level of liquid in a storage tank reaching some predetermined danger level.
- (d) They are both signal events, because they happen when a state transition occurs for whatever reason, such as a call or a time event. An *entry* event sets off an action sequence when a message causes a state to be entered. An *exit* event sets off an action sequence when a message causes a state to be left.

5.2 Looking inside states

When an object is in a particular state, either it remains idle waiting for some event to fire or trigger a transition, or it performs some activity while waiting. For example, a manufacturing robot might be used to move material around a factory. While it is in a *moving* state, it is likely to be monitoring its position at regular intervals on a given pathway. You have already seen how this kind of activity can be represented by use of the reserved word *do*, which was used in Figure 28. We could model the moving robot by adding an activity such as *do / monitorPosition* to the *moving* state.

As implied by their names, activities such as *monitorPosition* and *monitorTemperature* tell you that there is more detail within a given state. A state that contains no substructure is called a **simple state**, whereas a state that has other states nested within it, in order to represent some internal behaviour, is called a **composite state**. In a way, you have already seen examples of composite states, as you have seen how to model a state's internal events, such as the actions attached to entry and exit events. Indeed, an internal transition (see Subsection 4.3) implies that a given state will contain another state machine.

Recall the earlier telephone example in Figure 26, which shows two states: *idle* and *active*. You would expect the *active* state to be a composite state, since the main purpose of using a telephone is to call someone and have a conversation. Figure 29 shows a simplified model of a successful connection to another telephone. For simplicity, the parent state *active* is not shown. If the caller hangs up for any reason, the telephone will return to the *idle* state, which you can see in Figure 26, no matter which of the substates in Figure 29 it happens to be in when the *handset replaced* event occurs. Note too that any exit actions that may have been defined will also take place. For example, there may be exit actions defined for both the *talking* substate and the composite *active* state. If the *handset replaced* event occurs after a typical conversation, the exit action for the *talking* substate will be dispatched followed by that for the *active* state. As a consequence, there is no final state in the statechart diagram in Figure 29.

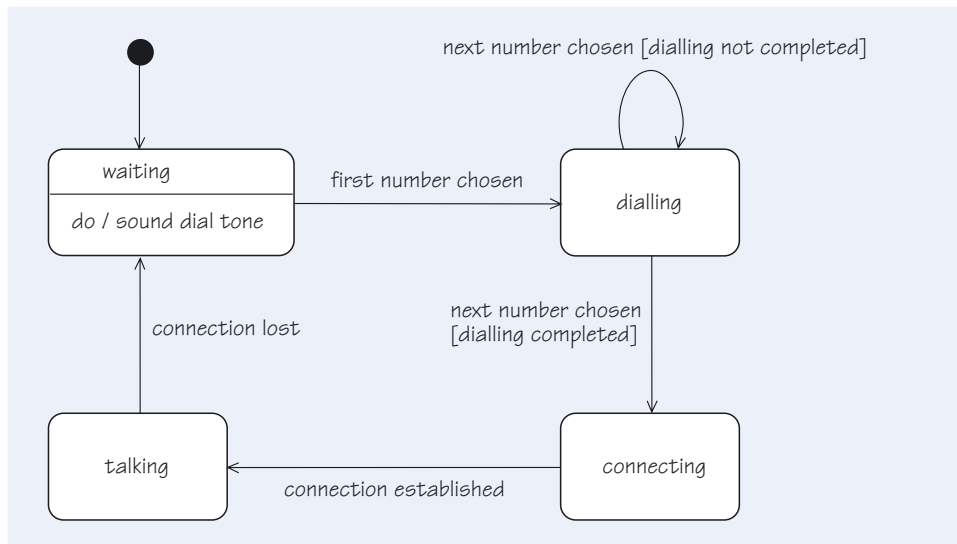


Figure 29 A statechart diagram of the composite state *active*

The *waiting* state in Figure 29 is the default substate that is entered on entering the *active* composite state. This is signified by using a transition from an initial state within the composite state. If you are using a default starting point within a composite state, then you should also make sure that all incoming transitions to the composite state terminate at the edge of the composite state. If you do not want to use a default state, UML allows you to have a transition from a state outside a composite state to a substate within the composite state. In such cases, the entry actions for the composite state are acted upon first, followed by the entry actions for the substate. Similarly, you can have a transition going from a substate within a composite state to a state outside the composite state.

UML allows you to label a composite state with the name of the state machine that models its internal behaviour, and you should use the reserved word *include* in the same way that you would add entry and exit events and their actions. On the *active* state in Figure 26, for example, you would add the following.

include / detailed active behaviour

In *Unit 3*, you saw that quite complex paths could be represented using decision boxes and synchronisation bars. State machines and activity diagrams are related, which means that you use the same notation in your diagrams:

- ▶ diamond-shaped boxes for the starts and ends of branches;
- ▶ thick horizontal or vertical lines to show synchronisation.

The use of a decision point is common where there is choice of states to enter after an object has been created or after a composite state has been entered. In the case of the lending library, for example, we can refine the simple state machine of Figure 22 (SAQ 7) to show that a particular copy of a book can be put on hold to fulfil a reservation. When a member brings back a particular copy of a book, a check should be made to see whether another member requires it. If there is no reservation for the book, the copy can be put back on the shelf; otherwise it is placed on hold. Figure 30 shows how we might revise our statechart diagram for a *Copy* object. Note that there is now a composite state called *in library*, which contains two substates, named *on shelf* and *on hold*. Each transition out of a decision point must be guarded to ensure that no more than one of them can fire on entry to the composite state.

Note that in comparison with Figure 26, the sounding of a dial tone has been moved from the transition *handset lifted* to an internal *do* activity. (Note that *do* does not indicate an internal transition.)

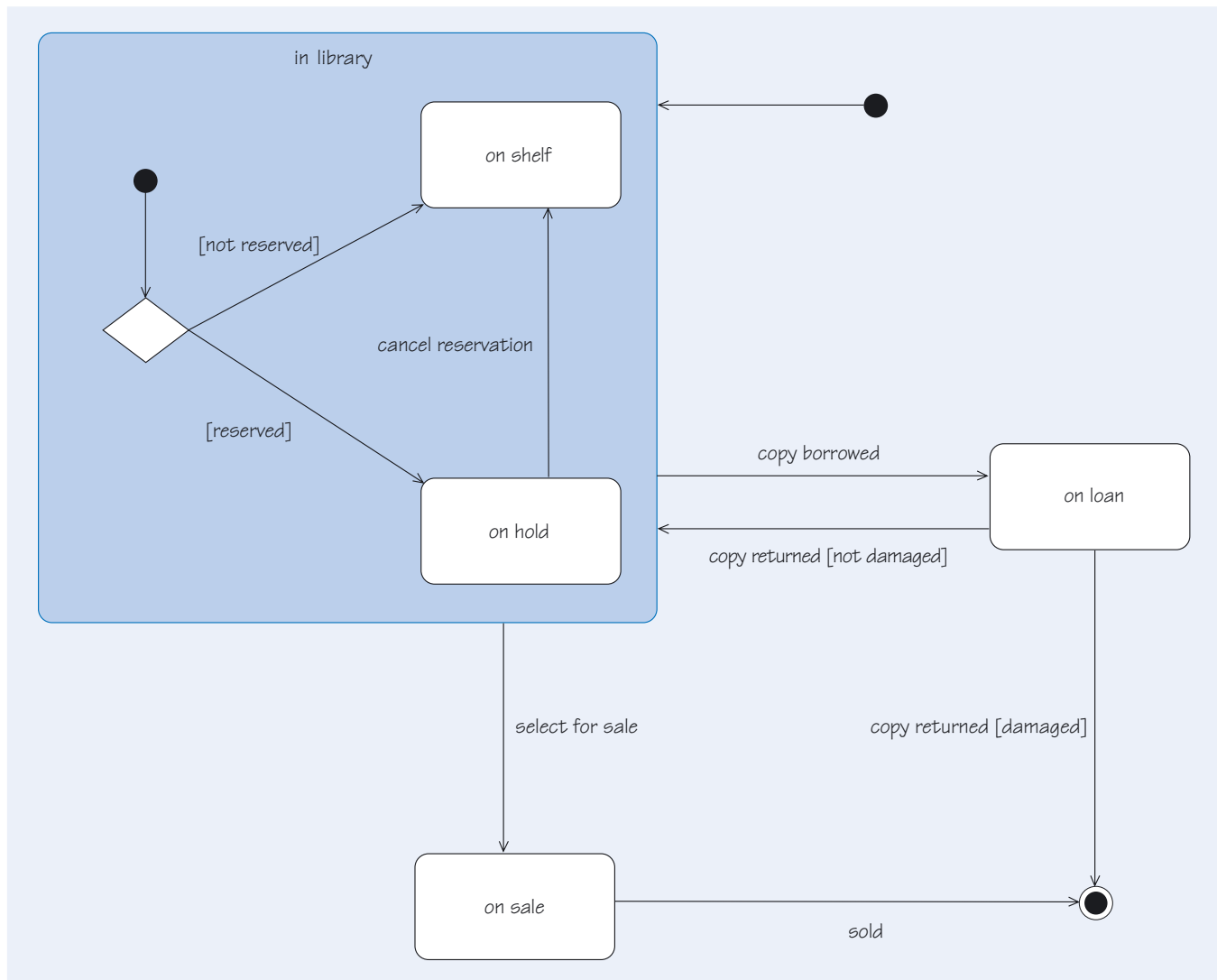


Figure 30 Adding detail to the life history of a *Copy* object

The composite states shown in Figures 29 and 30 are said to be *sequential* or *disjoint*. An object can be in only one of the substates while it is in the enclosing composite state. In contrast, you may want to model concurrent substates, that is, simultaneous situations within a composite state. A concurrent composite state is divided up into regions, separated by dashed lines, to represent the parallel behaviour. When an object enters a concurrent composite state, it also enters each one of the regions inside the composite state.

In practice, you are more likely to encounter sequential composite states.

SAQ 12

- What is a composite state in UML? Must composite states have initial and final states?
- Consider the *in library* composite state for the class *Copy* in Figure 30. A composite state contains one or more substates, but does the outer state change while the internal state machine is working? Use the example to explain your answer.

- ANSWER.....
- (a) A composite state is one that contains some internal behaviour, which can itself be represented as a separate state diagram. It is not compulsory to show an initial and a final state. However, you should use an initial state to show which substate is the default entry point.
 - (b) No. The outer state represents the condition of being in any one of the substates. When the *in library* state is entered, the *Copy* object will be in one of the substates shown in Figure 30. When a *Copy* object is in a substate such as *on shelf*, it is, by definition, also in the *in library* state. When the *Copy* object leaves a substate, it leaves the *in library* state.

5.3 A final note on state diagrams

It is possible to model quite complex behaviour within a single state diagram. However, such effort by the modeller must be justified, and one way to do that is to look for those objects that are dependent upon their internal state for their response to a given event. For example, if an object always responds in the same way to a given event, it is unlikely that you will need to produce a state diagram – it is **state-independent**. Those objects that do react differently to a given event are **state-dependent**. One useful clue is the need for some conditional logic for the success or failure of an operation, such as payment authorisation when you use a credit card in a supermarket.

In general, state-dependent behaviour is found in classes that must perform control and coordination tasks. For example, you will need to investigate state-dependent behaviour for software systems that control manufacturing processes and telecommunications, where there are a number of devices involved, such as switches, temperature sensors and valve controllers. Administrative systems can also have complex state-driven behaviour. For example, purchase and expense authorisation systems may have many levels of signature and approval, depending upon the amount involved.

You have already seen how interaction diagrams are used to model the flow of messages between objects, which are the most common form of internal event in the object-oriented paradigm. In other words, you can see how objects communicate in order to fulfil certain tasks. The need to model state-dependent behaviour is likely to be revealed as you model object collaborations, when, for example, you encounter the need to deal with conditional behaviour – is this credit card valid or not?

SAQ 13

What characteristics of events suggest that an object should respond to them when they occur?

ANSWER.....

An object should normally respond to:

- ▶ events that are external, such as those from a point of sale terminal;
- ▶ change or time events that require some response;
- ▶ certain changes such as high/low temperature, in process-control systems, to avoid problems or even disasters.



Exercise 7

Figure 31 shows a signal hierarchy for the kinds of signal that might arise from the use of a keyboard that is part of some system. Outline what would be needed to extend the hierarchy if a mouse were to be added to that system.

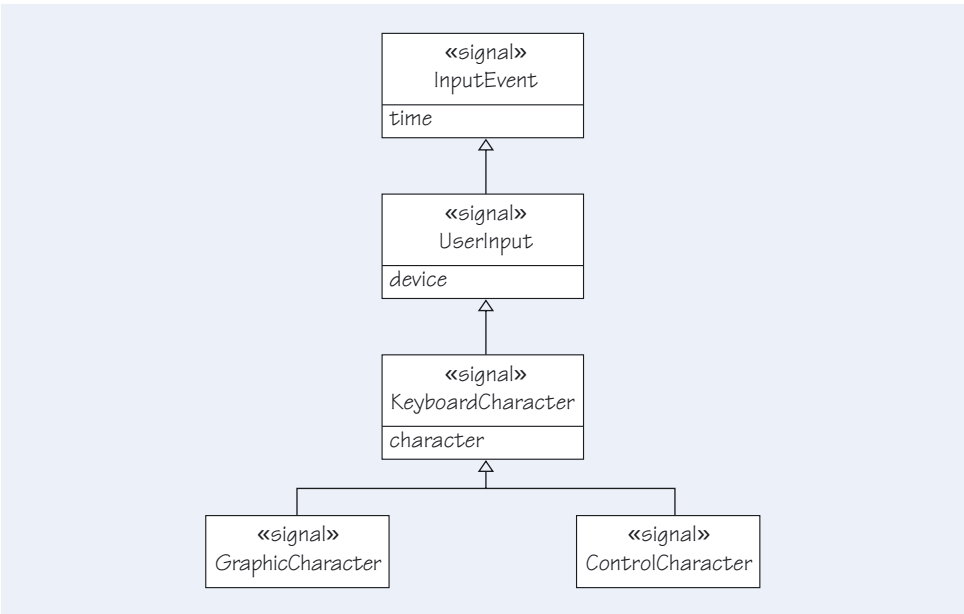


Figure 31 A model of keyboard signals as input events

Solution
You would need to add a series of signals to deal with the two-dimensional movement of the mouse. In the signal hierarchy, there would be a new device: *MouseButton*. You would need to record the signals from each button on the mouse. For each button, you might have two signals: *on/off* or *up/down*. Finally, you need to represent the signals from the roller ball on the mouse. (See also Exercise 1.)



Exercise 8

Figure 22 shows a statechart diagram for an object of the class *Copy* in the lending-library example. How might the diagram be amended to show that a copy becomes overdue if it is not returned within three weeks?

Solution
You can modify the statechart diagram in Figure 22 in a number of ways. The *Copy* object must still be in the *on loan* state. There are several choices, each one depending on a given library's rules for overdue books.
For example, if it is not necessary to record an overdue book because there is no penalty attached to it, you can use an internal transition or a self-transition that is triggered by a time event: *after(3 weeks) / notify member*. Which kind of transition you use would depend on the existence of any entry and/or exit actions.
If there is a penalty for being overdue, you might use a composite state for *on loan* that requires at least one substate, *overdue*, which is entered by a change event: *when (currentDate – issueDate > 3 weeks)*. Alternatively, you may defer calculating any fine until the book is returned, which would lead to an increase in the number of guarded transitions that respond to the *return* event.

Exercise 9



It is possible to model the behaviour of a complete system in a state machine, but it might not be desirable. If the system is simple enough, or if you can abstract enough detail away, you might use such a diagram to gain an understanding of a complex system. Draw a simple statechart diagram consisting of three states, *idle*, *heating* and *cooling*, that represents the behaviour needed to keep a room at a *constant* temperature. You can use the simple room-heater model in Figure 28 and a similar model for a refrigerator to model the more complex requirements of this exercise.

Solution

Figure 32 shows the transitions between *idle*, *heating* and *cooling* for a thermostatically controlled system to maintain room temperature. The notion of a *desiredTemperature* is shared between both the *heating* state and the *cooling* state of the model. To avoid the system thrashing between heating and cooling, we assume that *desiredTemperature* is represented as an integer (not a float), so that the system switches between heating and cooling if the current temperature is at least one degree away from the desired.

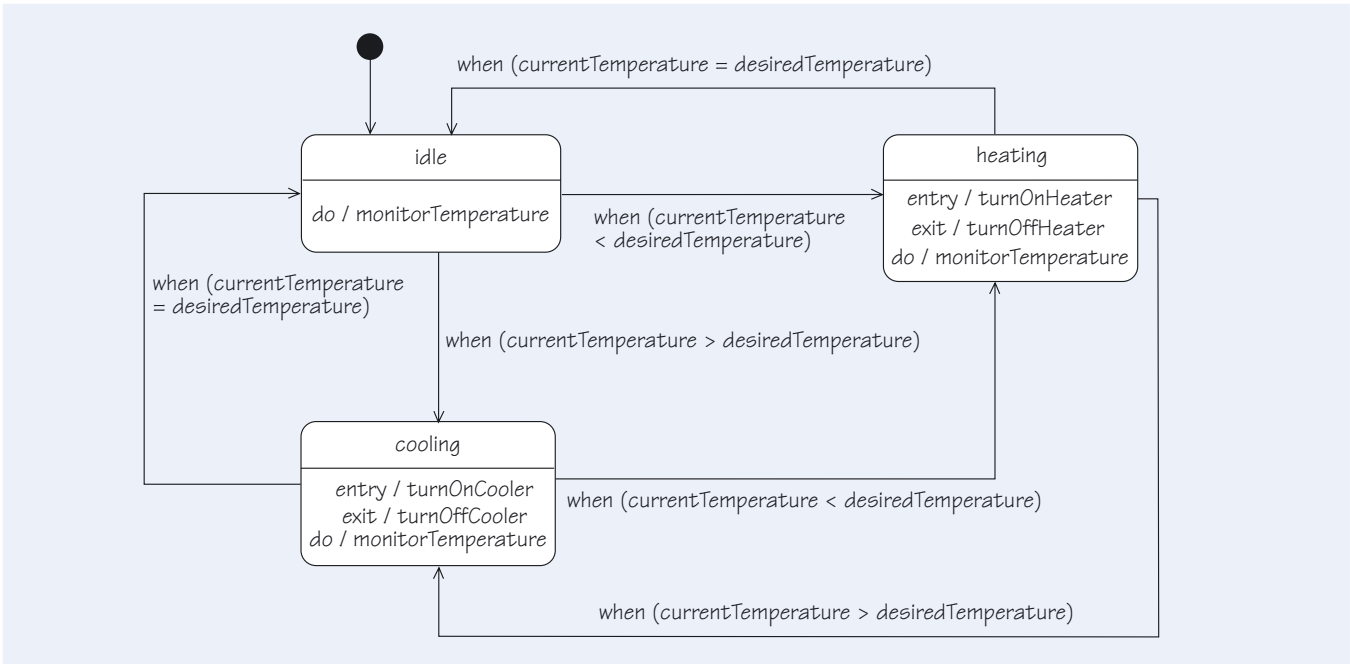


Figure 32 A state machine to keep a room at a desired temperature

In effect, the statechart diagram shows a combination of two simpler models: one for heating and one for cooling. The system returns to the *idle* state only when the *currentTemperature* matches the *desiredTemperature*. Until the *desiredTemperature* is reached or changed, the system is likely to alternate between *heating* and *cooling*.

Figure 32 has focused on the use of change events to support the comparison of current and desired temperatures. In practice, you may need to modify the expressions that are used in each change event to meet the accuracy of the devices used to measure the current temperature and set the desired temperature.



Exercise 10

UML lets us describe five things about a transition. What are they?

Solution

The five parts of a transition are:

- 1 source state;
- 2 target state;
- 3 event trigger;
- 4 action (or action sequence);
- 5 guard.

In practice, you may not need all five parts in every case.

5.4 Summary of section

In this section, you have learnt that it is possible to model quite complex behaviour in a state diagram. Apart from the familiar call events, such as message passing, there are also signal, time and change events in UML. A single state may contain a number of substates that can be modelled in one or more separate state diagrams. For example, a telephone in an active state may contain a number of substates that represent the life history of a call.

6 Packages

So far, we have concentrated on the refinement and elaboration of use case and class models to determine which operations belong to which class. Earlier in this unit, you saw ways of specifying the methods that comprise each operation within a class. In this section, we take a broader view of a software system as a whole.

UML provides a way of grouping model elements into a unit called a **package**. A UML package is a much more general idea than a Java package. A package can be used to group almost anything we please. Use cases can be grouped into packages, for example – and so can other packages!

Each package defines a **namespace**. Any two distinct model elements in the same namespace (package) must always have different names, although elements in different namespaces (packages) can have the same name. This is exactly the same idea that lets us have two Java classes with the same name, so long as they belong to different packages.

One package can be associated with another – if some of the model elements in the first package are associated with model elements in the second – and in a similar way a package can depend on another package. However, a package does not have an interface, and it cannot have instances, so packages cannot *interact* with one another.

There are three main reasons for partitioning a software system: to manage size and complexity; for information hiding; and for logical decomposition. It is common for businesses to operate management information systems (MIS) that consist of a number of separate packages to support their commercial activities. Figure 33 illustrates such a system, which contains four basic packages. It reflects a typical company that makes things (*Manufacturing*), which it stores (*Stock Control*), and then sells (*Order Processing*) to its customers. The income and expenditure are recorded in a series of transactions (*Accounts*). The notation for a package is a large rectangle with a smaller rectangle attached to its top edge – just like the tabs you use with the folders inside a filing cabinet. The name of each package can be written either inside the large rectangle, as in Figure 33, or inside the smaller of the two rectangles.

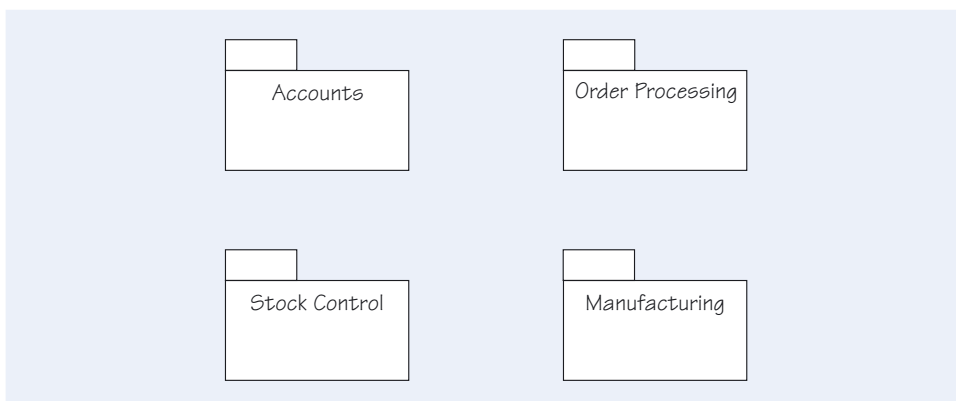


Figure 33 A typical management information system

Although Figure 33 does not show the dependencies between the packages, there will be a need for objects in each package to collaborate with one another. For example, customers place orders that must be made, delivered and then paid for. At the same time, the company pays for the raw materials and its employees' time taken to produce

By implication, there is a root package that contains the entire model of a software system.

Note the convention of removing the spaces between package names when creating unique names.

those orders. In UML, a package is not the same as a class, so it cannot have any associations or take part in any interactions. However, a class in one package can have an association with a class in another package – according to certain rules, as we explain below.

Since packages are a simple grouping mechanism for model elements, one package can contain other packages. You can either draw one package inside another, or represent the package hierarchy in a tree structure. For example, the *Accounts* package of Figure 33 may contain two other packages: one for accounts received, and the other for accounts payable (income and expenditure). Also, Figure 33 can be redrawn to show one large package, called *Management Information System*, containing the original four packages. However, the internal structure of each package is normally drawn in a separate diagram.

A package must have a unique name within the model. Each package forms a separate namespace, meaning that two model elements inside the same package cannot have the same name. However, model elements can have the same name if they are in different packages, so we should include the package name to distinguish between model elements. For example, you could place the class *Order* inside the *Order Processing* package in Figure 33, so that its unique name would be *OrderProcessing::Order*. Customers are typically part of the accounts received by a manufacturing company, so the class *Customer* would be identified as *Accounts::AccountsReceived::Customer*, which you can see in Figure 34.

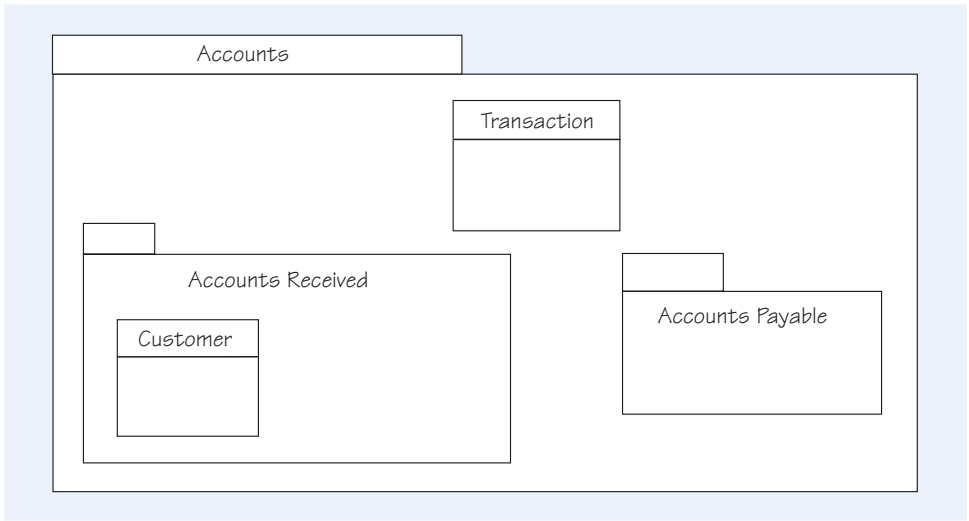


Figure 34 Packages can be nested inside each other

Once you partition your software system into packages, you need a way of allowing elements in different packages to ‘see’ each other.

The class *Customer* in Figure 34 has no difficulty in forming associations with any other class that is inside the same *Accounts Received* package, or with the class *Transaction*, since it is inside the same *Accounts* package that also contains the *Accounts Received* package. However, the class *Customer* is not able to ‘see inside’ the *Accounts Payable* package, because there is no shared namespace. In accounting terms, this barrier is quite helpful because you should not confuse income with expenditure. If you do want to overcome the namespace problem, UML allows you to import the contents of one package into another. In Figure 34, for example, you can draw a dependency arrow from *Accounts Received* to *Accounts Payable*, and label it with the stereotype «import». As a result, elements inside *Accounts Received* can refer to all the publicly visible elements of *Accounts Payable*, just as if they were all in the

UML allows you to mark individual elements of a package as public (+), private (–), protected (#) or package (~).

same package. The target package name does not need to be used for imported elements. Any private elements in *Accounts Payable* will remain unavailable.

An alternative mechanism is to use the stereotype «access», which defines a different form of dependency. Importing is a way of creating a larger namespace. In contrast, access dependency is similar to a 'special bridge' between the source and target packages. With access dependency, elements of *Accounts Received* can 'see' the public elements of *Accounts Payable*, for example. However, elements in the source package must include the target package name to access its elements, such as *Purchase* or *Supplier* in the accounting example.

If another package were to import or access the *Accounts Received* package, that package would not have access to elements of the *Accounts Payable* package. That package would need to either import or access the *Accounts Payable* package directly. In Figure 34, for example, *Order Processing* would need to explicitly import or access the contents of both *Accounts Received* and *Accounts Payable*, if there were a need to communicate with elements of both packages.

You can also use packages to present different views of a software system, such as its architecture. Figure 35 illustrates a three-layered architecture that we introduced in *Unit 1*.

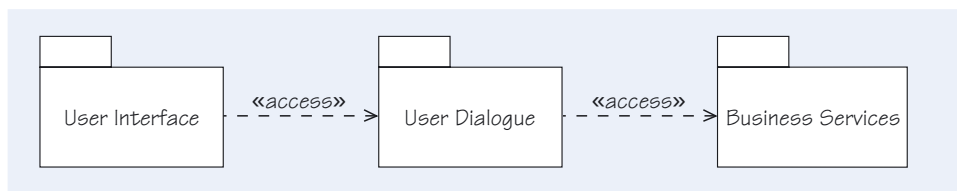


Figure 35 A three-tiered architecture

The user interface (representing the top layer in Figure 3 of *Unit 1*) translates user actions into system requests. In the MIS example of Figure 33, you would expect to see the classes that deal with the rules of order processing in the business services. These classes would then collaborate with the different classes of the user dialogue, such as the use case classes that you saw in Section 3. The separation of a software system into three layers or tiers has the following advantages:

- ▶ developers can be allocated to different layers to facilitate parallel development or the use of particular skills such as interface design;
- ▶ reuse becomes possible in certain areas, such as business services;
- ▶ distribution of the resultant software can be considered in response to certain performance requirements.

A well structured package will have a clear boundary and contain model elements that share a given purpose. That is, a well structured package displays high cohesion. In addition, the contents of a package and their visibility should be such that other packages can only see those elements that they need in order to do a specific job. That is, a well structured package displays low coupling. As you saw in *Unit 1*, high cohesion and low coupling are prerequisites for useful components. However, a package lacks several things: it has no interface, cannot be instantiated, and cannot participate in interactions. Subsystems, which we look at next, remedy these deficiencies.

Packages also define something that can be tested, as you will see in *Unit 11*.

SAQ 14

- (a) Do packages only contain classes?
- (b) Which modelling principle would lead you to form packages?

- (c) Explain how it is possible to have two separate elements with the same name.
- (d) The stereotypes «*import*» and «*access*» allow you to 'see' through the 'wall' around a package. Which one can give rise to a naming problem? Which one provides the simpler traceability?
- (e) Suppose you used «*access*» to 'see' into a package. Can you see everything inside that package? Briefly explain your answer.

ANSWER.....

- (a) No – they can contain any model element from UML. Among other things, you can include classes, use cases, associations and even other packages.
- (b) You should use a package to raise the abstraction level, suppressing any detail in a particular diagram that is not necessary to understand what the model is showing.
- (c) If you place the two elements in separate packages, they can have the same name. This is because each model element is identified by its name and the name of the smallest package that contains it.
- (d) The «*import*» stereotype can give rise to a naming problem, because an «*import*» adds the names of the elements in the target package to the source package. The result is that, if an imported name clashes with an existing name, you have to qualify the imported elements with their package name in order to distinguish them. The «*access*» stereotype allows simpler traceability, because other packages must use the full name of an accessed element, such as *PackageName::ElementName*.
- (e) There is no guarantee that you can 'see' all the elements of the package. It is possible to control the visibility of elements within a package in the same way that you can control the visibility of the attributes of a class. An element of the accessed package can be marked as private, making it invisible to other packages that import this package and indicated with a '-' character. Alternatively, it can be marked as protected, making it invisible to importers of the package but visible to child packages, and indicated with a '#' character.

6.1 Subsystems

Since packages are not classifiers in UML, they cannot be instantiated – they are not visible at run-time. A subsystem is a special kind of package that *can* be instantiated. In addition, a subsystem has two sets of elements, one for its specification and the other for its realisation. Use cases are the main element for the specification side, while classes define the realisation side. You can also add the interfaces that the subsystem conforms to, just as you would on a class diagram.

A subsystem is indicated by adding either the «*subsystem*» stereotype or the fork symbol to the tab where the name of the package appears. The fork symbol is shown in Figure 36.

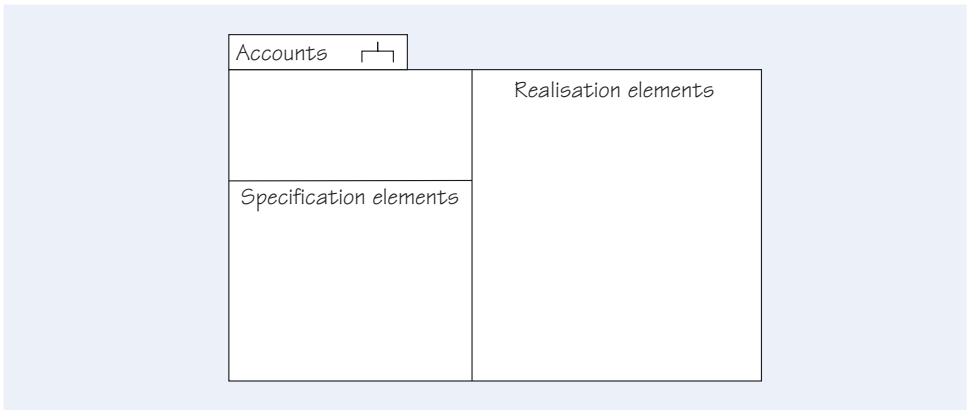


Figure 36 The basic notation for a subsystem

The unnamed compartment in Figure 36 is used to identify the operations of the subsystem. It is easy to see that a subsystem diagram can become somewhat busy, which means that a CASE tool is generally beneficial in helping the developer keep track of the complexity. In practice, you do not need to use all three compartments, although your chosen development process may define how you should use the notation.

There is a close correspondence between subsystems and the components that you will study in *Unit 10*, because a component can be viewed as an implementation of a subsystem – according to the specification of that subsystem, and constrained by its realisation elements (design).

SAQ 15

- (a) What is a subsystem?
- (b) What is the structure of a subsystem in UML?

ANSWER.....

- (a) A subsystem is a package of elements that are treated as a single unit or model element. It is a coherent piece, or partition, of a system that is treated as a single abstraction. In effect, it is modelled as both a package and a classifier. It is a solution to the common problem of wanting a package to take part in interactions.
- (b) A subsystem contains a behavioural specification and the things needed to implement that behaviour. There are three groups of elements in UML. One group contains specification elements such as use cases, constraints and the relationships between them. A second group contains the operations and, by implication, interfaces needed to meet the specification. A third group contains the elements needed to implement the behaviour defined by the other groups. Thus the third group will contain classes, associations and named collaborations or patterns.

Exercise 11

A typical accounting system consists of three distinct concerns:

- 1 accounts received, which represents a company’s income;
- 2 accounts payable, which represents a company’s expenditure;
- 3 nominal or general ledger, which a company uses to reconcile income and expenditure.

Redraw the accounts package of Figure 33 to illustrate these different concerns.



Solution

Figure 37 shows one way to partition the accounts package of a typical MIS.

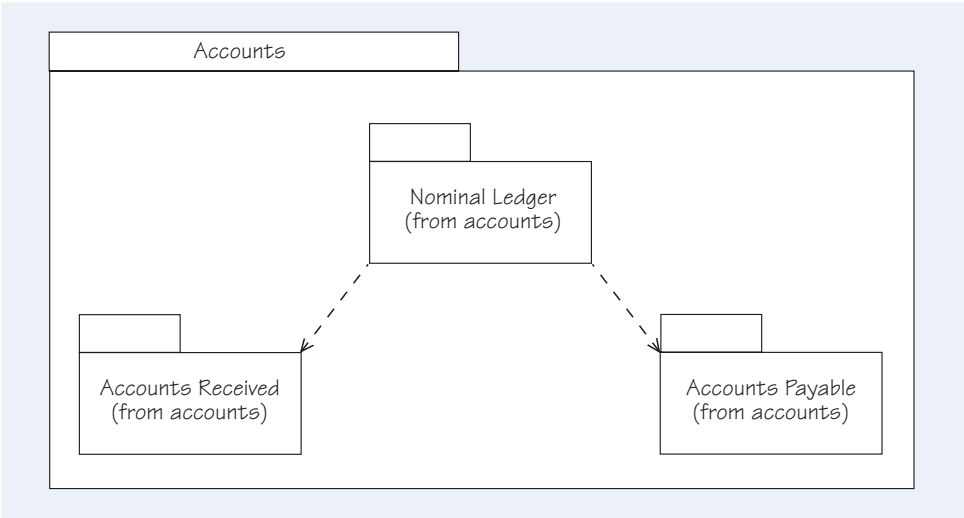


Figure 37 Accounting packages

We have shown only the basic dependency of the *Nominal Ledger* upon the *Accounts Received* and *Accounts Payable* packages. In practice, there will be more dependencies, and you would need to decide whether import or access dependency would be used in each case.



Exercise 12

Classes are just one of the elements you might choose to gather together into a package. List four reasons, other than simple convenience, for collecting model elements into a named package.

Solution

Packages are a way of partitioning models and their elements that are:

- 1 closely related, such as those in the same subject area or domain;
- 2 strongly associated, such as a specific collaboration;
- 3 in a hierarchy, such as generalisation/specialisation;
- 4 participating in the same use cases.

In general, you should be looking for some aspect of shared purpose, concepts or behaviour.



Exercise 13

Packages and classes are model elements. They are both abstractions to help understand some aspect of a system. However, a package is not a classifier, whereas a class is. Suggest an important distinction between the packages and classes that you might represent in a model. How does a subsystem get around this distinction?

Solution

In practice, you are likely to identify both classes and packages in the same model. You would use classes to represent the abstractions of the things you found in the problem or solution domain. Packages are simply a means of organising model elements, perhaps to remove detail to improve understanding. Packages cannot be

instantiated: they do not respond to messages or have instances. In contrast, classes can be instantiated: they have instances that respond to messages. The objects are the elements of an operational software system.

A subsystem is a special kind of package. It is instantiable. Because a subsystem has an interface, can be instantiated, and can represent behaviours, it is in effect a cross between a package and a class.

6.2 Summary of section

In this section, you have learnt that model elements of almost any sort can be grouped into coherent units called packages.

- ▶ Packages help you organise models in a way that lets you better represent and communicate your understanding of a problem.
- ▶ You can assign a dedicated team to each package during a development project.
- ▶ Packages can help you construct large software systems by splitting them up into smaller, more manageable chunks.

A package defines a namespace. No two elements in the same package may have the same name, but elements in different packages may have the same name and are distinguishable by their respective package names.

An element inside a package is only visible from other packages if it is declared public. For the element to be visible from another package, it must be imported to that package, using either *«import»* or *«access»*.

Packages can be associated with, or depend upon, other packages. Ideally, packages should exhibit high cohesion and low coupling.

Packages are not classifiers: they cannot have instances, do not have interfaces, and cannot take part in interactions. A special kind of package called a subsystem overcomes these restrictions and allows us to describe what the subsystem should do. Subsystems are closely related to software components.

7

Summary

As you get closer to implementing your design, it is likely that you will alternate between static and dynamic modelling in revising your design to satisfy the users' requirements. It is possible to follow each scenario using a set of sequence diagrams, so that each one illustrates what will happen in a given set of circumstances for a given set of objects. You may then need to modify your class model as you try to find the best software solution to the problem. In practice, there will be a number of design issues that influence the kind of software system that you can implement.

A good model will need to be:

- ▶ a correct representation of the requirements; otherwise your effort will be wasted during the implementation activity;
- ▶ consistent; otherwise the teams of developers on a large project will not be able to integrate the packages on which they have worked;
- ▶ sufficiently complete; otherwise you may fail to model an important requirement;
- ▶ well structured by being modularised into packages.

In practice, you will experience a tension between the need to produce a good model and the needs of the project that you are working on. Models alone are not enough to produce a good software system, as you will see in *Unit 9*.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ use UML notation for message sends in sequence diagrams;
- ▶ express simple concurrent designs in UML;
- ▶ use some UML notations for expressing timing constraints;
- ▶ identify alternatives that influence where you might place the checking of the preconditions for an operation;
- ▶ discuss the merits of three strategies for the implementation of use cases;
- ▶ identify the main components of a state machine;
- ▶ identify when a state machine can contribute to the development of software;
- ▶ describe the different kinds of event used in UML;
- ▶ describe a way of controlling the visibility of one or more elements to other elements in the same model;
- ▶ explain how packages promote modularisation within a large software system.

Index

A

actions 27, 32
active classes 10
active objects 10
activities 32
activity, of objects 10
actor classes 20

C

call events 39
change events 39
Command pattern 20
composite states 42
concurrency 10
conditional behaviour 6
conditional message sends 6

D

defensive programming 18
Design by Contract 18

E

entry events, in state machines 32
events 24
 entry and exit 32
 external and internal 39

exit events, in state machines 32
external events 39

F

final states, in statechart diagrams 34

G

guards 6, 25, 29

I

initial states, in statechart diagrams 26
internal events 39
internal transitions 33
iterative message sends 8

M

mailbox semantics 12
management information systems (MIS) 49
MIS (management information systems) 49

N

namespaces 49

P

packages 49

passivity, of objects 10

postconditions 18

preconditions 18

R

rendezvous semantics 12

S

self-transitions 29
signal events 39
simple states 42
state machines 24
state-dependence 24, 45
state-independence 45
statecharts 24
states 24
 composite 42
 final 34
 initial 26
 simple 42
subsystems 52

T

time events 39
timing 13

U

use case objects 20

